FULL WALKTHROUGH

The MRC Operations System — Every Path, Every Button, Every Edge Case

A ground-truth tour of the MRC operations system — what's built, what's deliberately not, and what's still a gap. Three role journeys, four cross-cutting references, five appendices. Written so three different readers (admin, co-owner, auditor) find what they need without context-switching.

Built for: a new admin on Monday morning · a co-owner setting priorities · an auditor or acquirer verifying operational maturity

Companion to: `docs/mrc-system-overview.html` (executive summary)

0. How to read this

The document has three parts.

Journeys (1–3). How each role — customer, technician, admin — moves through the system, in the order things happen.

Cross-cutting references (4–7). What touches every journey: notifications, failure modes, permissions, and the full list of screens.

Appendices (8–12). Reference material for Edge Functions, the database, integrations, known issues, and a glossary.

Status tags:

- ✓ BUILT — implemented and working in production.
- ⚠ PARTIAL — scaffolding exists; key pieces missing or stubbed.
- ✕ NOT BUILT — planned or assumed but not implemented.
- ⊖ DELIBERATE — intentionally not built. Examples: customer cancel button (retention phone call is the product), self-service portal (not yet planned), SMS sending (cost / compliance), refunds (handled outside the system).
- i NOTE — non-obvious behaviour or archaeology.

Sections marked DEV are denser — Edge Functions, database schema. Skim them safely if you're not technical.

Every file path is real. Every quoted string was read out of the code at the cited line.

★ Day One — the four things that matter most

If you're new to the business or to the system, internalise these four before anything else. They explain ninety percent of the surprises a new admin will hit in their first week.

- **Phone calls fill every gap the app leaves.** The app does not handle cancellation, dispute, payment chasing, or out-of-hours communication. When a customer wants to change something, they ring the office. The app records the result of that call after the fact.
- **Technicians learn about new jobs by opening the app.** There is no push notification, no SMS, no email when a job is assigned. If you assign a tech to a job, you also need to text or call them — the app will not.
- **The 13% discount cap is enforced at the database.** It is not a UI convention. The `invoices` table has a CHECK constraint that refuses any value above 13%. There is no override. Any pricing exception above 13% requires a code and database change.
- **Slack is the system's nervous system.** Every status change, every booking, every payment, every overdue milestone posts to the team Slack channel. If something happened in the app and it didn't appear in Slack, it didn't really happen — start your investigation there.

⭐⭐ Before Real Use — pre-launch checklist

Action items that must be ticked off before MRC switches off the test setup and routes real customer traffic through this system. Each line is operational, not aspirational. The groups are roughly sequenced by dependency — work top-down for safety.

Domain & Framer cutover

- [ ] Switch the Framer webhook source from the test site (`fluid-members-390105.framer.app/contact`) to the real production Framer site.
- [ ] Apply the Framer field rename (docs/FRAMER_FIELD_MAPPING.md) on the **real** Framer site too — not just the test site.
- [ ] Verify the real Framer form has all 9 fields named correctly: `full_name` , `phone` , `email` , `street` , `suburb` , `postcode` , `preferred_date` , `preferred_time` , `issue_description` .
- [ ] Confirm the real site's webhook destination is set to `https://ecyivrxjpsmjmexqatym.supabase.co/functions/v1/receive-framer-lead` .
- [ ] Confirm the real form's Fallback email is set to `admin@mouldandrestoration.com.au` .
- [ ] Submit one live test lead through the real Framer site post-switch and verify it lands in `webhook_submissions` as `status='processed'` with a flat payload.

Email domain

- [ ] Set up DNS records (SPF, DKIM, DMARC) for `mouldandrestoration.com.au` on the actual DNS provider.
- [ ] Verify the domain in the Resend dashboard.
- [ ] Update `send-email` , `send-inspection-reminder` , `receive-framer-lead` Edge Functions to send *from* `noreply@mouldandrestoration.com.au` (currently `noreply@mrcsystem.com`).
- [ ] Update reply-to across all email infrastructure to `admin@mouldandrestoration.com.au` .
- [ ] Redeploy the three Edge Functions after the change.
- [ ] Send one test email end-to-end (booking confirmation or inspection report) and verify it arrives, reply-to is correct, and the SPF/DKIM/DMARC headers pass.

Account & billing ownership transfer

- [ ] Supabase project — transfer ownership to the business account.
- [ ] Vercel project — transfer ownership to the business account.
- [ ] Resend account — transfer to the business account (or move the verified domain to it).
- [ ] Sentry organization — transfer to the business account.
- [ ] Slack workspace — transfer to the business account, or add the business as a workspace admin.
- [ ] OpenRouter / Gemini — create API keys under the business account; swap the new keys into Edge Function env vars (`OPENROUTER_API_KEY`) and revoke the old personal keys.
- [ ] Google Maps — create the API key under the business Google Cloud account; swap into Edge Function env vars (`GOOGLE_MAPS_API_KEY`) and revoke the old key.

Data migration (ServiceM8 → MRC)

- [ ] Export leads, contacts, jobs, and invoices from ServiceM8.
- [ ] Write a one-time migration script mapping ServiceM8 fields to the MRC schema (`leads` , `inspections` , `job_completions` , `invoices`).
- [ ] Run the migration against a dev/staging Supabase project — **not** production.
- [ ] Verify all migrated leads render correctly in the admin UI (contact info, addresses, status, dates).
- [ ] Handle duplicates between ServiceM8 data and any existing rows already in MRC (use the new `is_possible_duplicate` + `possible_duplicate_of` flag pattern, or a simpler email+phone match).
- [ ] Backfill the 8 historical empty-postcode rows — manual admin lookup (phone the customer, or Google the suburb's canonical postcode).

- [ ] Once verified on dev, run the migration against the production Supabase project.
- [ ] Archive or revoke ServiceM8 read-only access once parity is confirmed.

P0 backlog (must close before real use)

- [ ] Set up a separate test copy of the database. Right now, any database change the developer makes is applied directly to the live system that holds real customer data — there's no safety net. Creating a second "practice" database means changes can be tested there first before they touch real customers. One evening of setup work. (See Section 11.)

Operational readiness

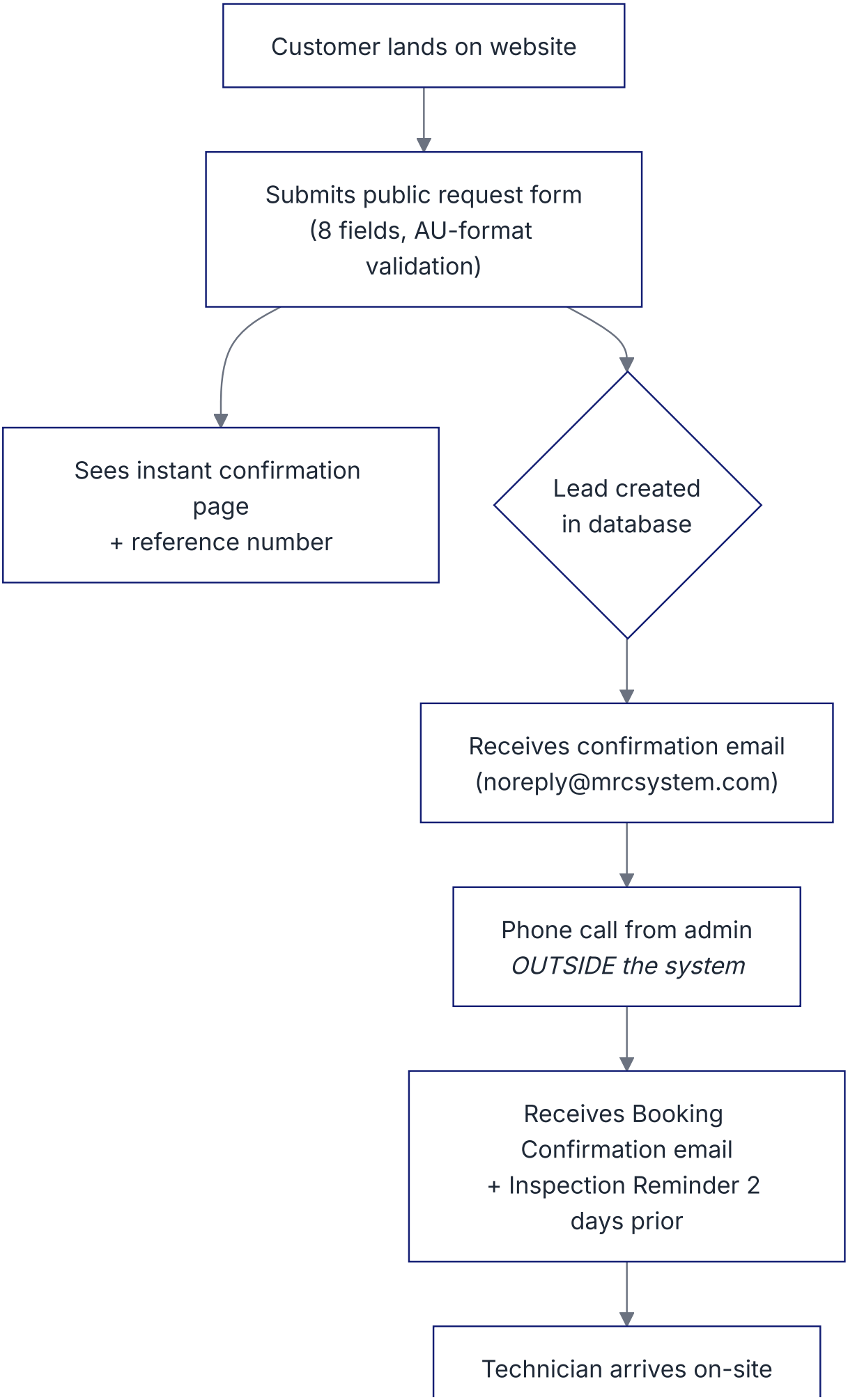
- [ ] Confirm Glen and Clayton have tested their respective flows end-to-end on real devices (their actual phones, not a desktop browser).
- [ ] Confirm Michael (admin) has tested the full lead → finished cycle on production with a real test customer.
- [ ] Confirm the Slack `#leads` channel is live, the team is in it, and notifications are not muted.
- [ ] Confirm the overdue-invoice cron is running on schedule (`0 23 * * *` UTC = 09:00 AEST). Verify by checking the most recent run in Supabase logs.
- [ ] Confirm the `webhook_submissions` table is being monitored — any row with `status='failed'` in the last 7 days surfaces on the Admin Dashboard "Failed Webhooks" KPI.
- [ ] Define a rollback plan. If real use breaks, how is it reverted? Is ServiceM8 still available as a fallback? How fast can the team go back to it?

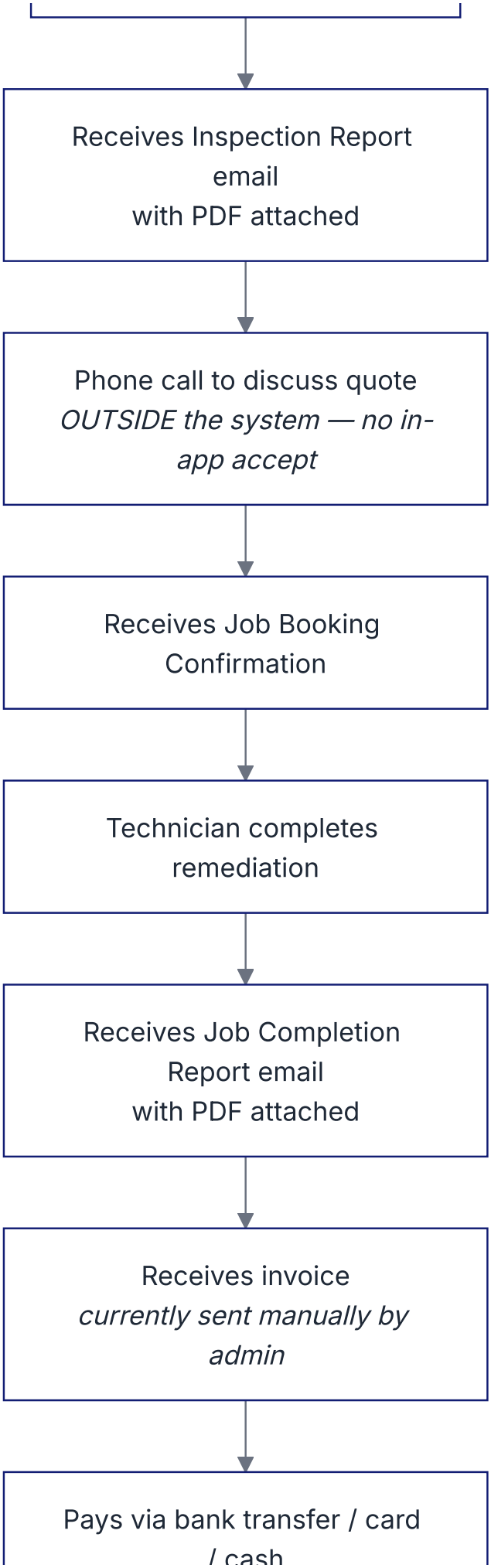
Post-launch monitoring (first 7 days)

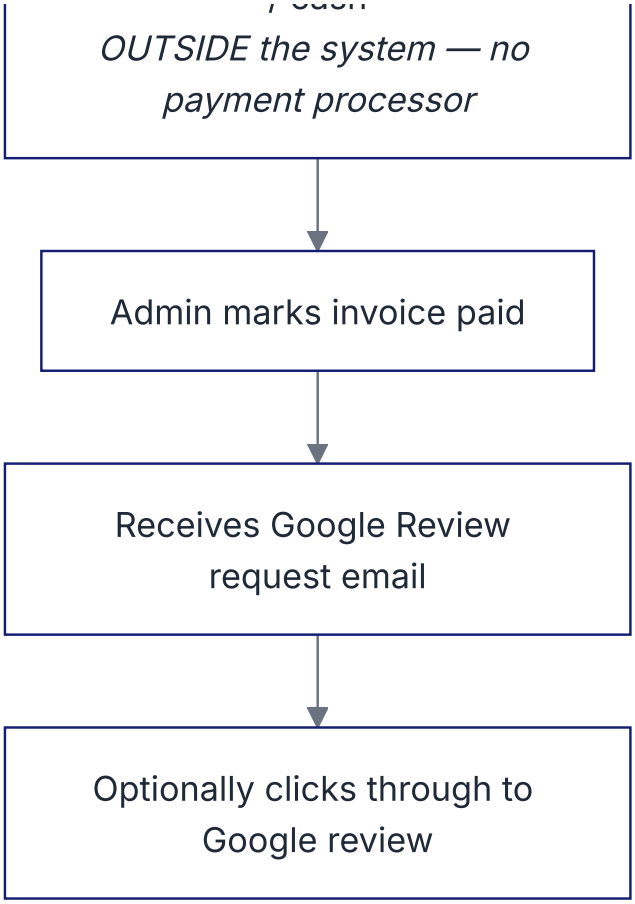
- [ ] Watch the Sentry error rate for the first 72 hours. Spike means investigate immediately.
- [ ] Watch Slack `#leads` for missed or dropped leads — every webhook submission should produce both a Slack alert and a row in the database.
- [ ] Watch the Resend dashboard for bounce rate. Anything above 2–3% needs attention.
- [ ] Check the `webhook_submissions` status distribution daily for the first week. Mostly `'processed'`; the occasional `'rate_limited'` is fine; `'failed'` rows need investigation.
- [ ] Log every issue Glen, Clayton, or Michael hits, and triage fast — week-one issues set the team's confidence in the platform.

1. Journey 1 — The Customer

The customer never logs in. Their entire relationship with MRC happens through a single public form, a series of emails, and (mostly) phone calls. Below is every touchpoint they experience in order, with notes on what is genuinely automated vs handled by an admin behind the scenes.


Customer lands on website
Submits public request form (8 fields, AU-format validation)
Sees instant confirmation page + reference number
Lead created in database
Receives confirmation email (noreply@mrcsystem.com)
Phone call from admin OUTSIDE the system
Receives Booking Confirmation email + Inspection Reminder 2 days prior
Technician arrives on-site


Receives Inspection Report email
with PDF attached
Phone call to discuss quote
OUTSIDE the system — no in-app accept
Receives Job Booking Confirmation
Technician completes remediation
Receives Job Completion Report email
with PDF attached
Receives invoice
currently sent manually by admin
Pays via bank transfer / card / cash



Customer journey. The app is the office's tool, not a customer self-service portal — that will be planned moving forward, once real usage data tells us what customers actually need.

1.1 The public request form

✓ BUILT The public intake is the Framer marketing site's "Request Inspection" form. It posts to the `receive-framer-lead` Edge Function (`supabase/functions/receive-framer-lead/index.ts`); see Section 8 for the function summary and Section 10.8 for the Framer integration overall.

What the form captures:

- **Full Name** required — minimum 1 character.
- **Phone Number** required — minimum 8 digits.
- **Email** required — standard email validation.
- **Street Address** required
- **Suburb** required
- **Postcode** required — Melbourne range, validated 3000–3999, written to `property_address_postcode` .
- **Preferred Date of Booking** required — free-form date string, normalised to ISO.
- **Preferred Time of Booking** required
- **Issue Description** required — textarea.

Action for Michael: confirm in the Framer form editor that these three inputs have Required = Yes. If not, flip them on and republish. The `receive-framer-lead` Zod schema (`supabase/functions/receive-framer-lead/index.ts:61-72`) currently marks them optional; a future commit may want to tighten the schema too.

Validation runs against a Zod schema in the Edge Function (`receive-framer-lead/index.ts:61-70`). The function smart-parses the incoming payload — it accepts JSON, multipart form-data, and url-encoded bodies, and falls back to regex-based content detectior (email / phone / date / time patterns) for any field whose name doesn't match the expected keys (`:501-561`). This makes the function robust against Framer field-name changes.

> **Customer-preferred date and time are not confirmed bookings.** The Preferred Date and Preferred Time inputs land in the `inspection_scheduled_date` and `scheduled_time` columns on the leads table — the same columns that hold real, admin-confirmed bookings. The Admin Dashboard "Today's Jobs" KPI and the Schedule view treat them identically. Admin must reconfirm before treating any of these as actual appointments. Tracked as P2 in Section 11.

1.1a Admin manual entry

✓ BUILT When a customer phones the office, or a lead arrives from hipages or another non-website channel, an admin enters it manually via the "New Lead" button on the Admin Dashboard or the Leads Management screen. Source: `src/components/leads/CreateNewLeadModal.tsx`.

The modal differs from the Framer form in three ways. It uses Google Maps autocomplete for the address and captures latitude/longitude (`CreateNewLeadModal.tsx:4` imports `useAddressAutocomplete`), which then drives the `property_zone` calculation via `calculatePropertyZone(lat, lng)` . It offers a date picker plus 30-minute time slots from 7 AM to 6 PM (`:82-91`). And it lets the admin pick a free-text `lead_source` from a dropdown (`leadSourceOptions`) rather than the channel-specific value the Framer path writes.

1.2 What happens when a Framer submission arrives

✓ BUILT The Edge Function processes every webhook in this exact sequence:

1. **Raw body logged.** Before any parsing, the raw payload is written to the `webhook_submissions` table with status `'received'` (`:429-435`). This is the audit-survives-everything log: even if the function later fails, the raw payload is preserved for manual recovery.
2. **Body size + rate-limit checks.** Bodies over 50 KB are rejected (413). IPs that have submitted more than 5 times in the last hour are rejected (429). Both update `webhook_submissions.status` to `'failed'` or `'rate_limited'` respectively (`:11-12, 450-475`) and fire admin alert email + Slack.
3. **Body parsed.** Supports JSON, multipart form-data, and url-encoded content types (`:483-499`).
4. **Smart field extraction.** Tries known field-name aliases first (e.g. accepts both `full_name` and `"Full Name"`); then falls back to regex content detection on remaining values (email pattern, phone pattern, date pattern, time pattern) for any unmatched fields. Robust against Framer field renames (`:501-561`).
5. **Zod validation.** Failure inserts a `'failed'` `webhook_submissions` row, fires admin alert email + Slack, returns 400 (`:582-593`).
6. **Duplicate check.** Same email + phone within the last 24 hours returns 200 with "Lead already received" message and updates the submission status to `'duplicate'` (`:595-607`).
7. **Insert with retry.** Three attempts, exponential backoff (500 ms → 1500 ms → 4500 ms). Three failures triggers admin alert email + Slack and returns 500 (`:364-383, 623-637`).
8. **Fire-and-forget notifications.** On success: Slack Block Kit message to the team channel ("🏠 New Lead Received"), confirmation email to the customer (branded MRC template, sent via Resend directly — does not go through the `send-email` Edge Function), `email_logs` row inserted with template name `'framer_lead_confirmation'` (`:653-698`).

The submission-status column on `webhook_submissions` feeds the "Failed Webhooks" KPI on the Admin Dashboard. Any row with status `'failed'` in the last 7 days surfaces there.

1.3 Every email the customer receives

All emails are sent via Resend. Templates live in `src/lib/api/notifications.ts` (with one inline template in `supabase/functions/receive-framer-lead/index.ts` for the public-form path, and the deployed reminder template in `supabase/functions/send-inspection-reminder/index.ts`). All seven share a branded wrapper (navy header, gold accents, signature block with phone, email, hours, Google review link).

> **Sender and reply-to addresses still use `@mrcsystem.com`** ; full domain cutover is tracked in ★★ Before Real Use. The displayed contact email inside the email signature (and in PDF reports) was switched to `admin@mouldandrestoration.com.au` ahead of the rest of the cutover.

#	Email	Trigger	Attachment	Action expected
1	**Enquiry confirmation**	Form submitted	None	None — wait for callback
2	**Booking Confirmation**	Admin schedules inspection	None	Ensure property access; reschedule by phone
3	**Inspection Reminder**	Cron, ~2 days before booking	None	Confirm access; reschedule by phone
4	**Inspection Report**	Admin sends from email-approval screen (after approving the draft)	PDF attached (inspection report PDF)	Read; phone to discuss quote
5	**Job Booking Confirmation**	Admin schedules remediation job	None	Ensure access; reschedule by phone
6	**Job Completion Report**	Admin sends from email-approval screen (after approving the draft)	PDF attached (job completion report with before/after photos + warranty)	Read; reply with any issues
7	**Google Review Request**	Admin clicks "Send Google Review" after payment	None	**Click the Google review link** (only clickable CTA in any customer email)

Preview gallery

Real email screenshots for each of these 7 templates are in the MRC Google Drive. The preview metadata below (subject, sender, reply-to, template name, and source file) is the authoritative reference for what builds each email.

#1 — Enquiry confirmation

Subject: Thank you for your enquiry - Mould & Restoration Co
From: `noreply@mrcsystem.com` · **Reply-to:** `admin@mrcsystem.com` · **Template name:** `framer_lead_confirmation`
Builder: `supabase/functions/receive-framer-lead/index.ts:204`

Preview: *Real email screenshot stored in the MRC Google Drive, alongside this document.*

#2 — Booking Confirmation

Subject: Booking Confirmed — 15/05/2026 at 10:00 AM
From: `noreply@mrcsystem.com` · **Reply-to:** `admin@mrcsystem.com` · **Template name:** `booking_confirmation`
Builder: `src/lib/api/notifications.ts:185`

Preview: *Real email screenshot stored in the MRC Google Drive, alongside this document.*

#3 — Inspection Reminder

Subject: Inspection Reminder — 2 days away
From: `noreply@mrcsystem.com` · **Reply-to:** `admin@mrcsystem.com` · **Template name:** `inspection_reminder`
Builder: `supabase/functions/send-inspection-reminder/index.ts:92` (cron-deployed)

Preview: *Real email screenshot stored in the MRC Google Drive, alongside this document.*

#4 — Inspection Report (approved)

Subject: Your Inspection Report is Ready — MRC-2026-0138
From: `noreply@mrcsystem.com` · **Reply-to:** `admin@mrcsystem.com` · **Template name:** `report-approved` · **Attachment:** inspection report PDF
Builder: `src/lib/api/notifications.ts:205`

Preview: *Real email screenshot stored in the MRC Google Drive, alongside this document.*

#5 — Job Booking Confirmation

Subject: Job Booking Confirmed — MRC-2026-0138
From: `noreply@mrcsystem.com` · **Reply-to:** `admin@mrcsystem.com` · **Template name:** `job_booking_confirmation`
Builder: `src/lib/api/notifications.ts:252`

Preview: *Real email screenshot stored in the MRC Google Drive, alongside this document.*

#6 — Job Completion Report

Subject: Job Completion Report — JOB-2026-0042
From: `noreply@mrcsystem.com` · **Reply-to:** `admin@mrcsystem.com` · **Template name:** `job_report_sent` · **Attachment:** job completion report PDF (filename `MRC-JOB-{jobNumber}-Job-Report.pdf`)
Builder: `src/lib/api/notifications.ts:329`

Preview: *Real email screenshot stored in the MRC Google Drive, alongside this document.*

#7 — Google Review Request

Subject: Thank you from Mould & Restoration Co
From: `noreply@mrcsystem.com` · **Reply-to:** `admin@mrcsystem.com` · **Template name:** `google_review_request`
Builder: `src/lib/api/notifications.ts:411`

Preview: *Real email screenshot stored in the MRC Google Drive, alongside this document.*

Subject lines and body text are built dynamically per template — see functions `buildBookingConfirmationHtml()` , `buildInspectionReminderHtml()` , `buildReportApprovedHtml()` , `buildJobBookingConfirmationHtml()` , `buildJobReportEmailHtml()` , `buildGoogleReviewEmailHtml()` in `src/lib/api/notifications.ts` .

1.4 How the customer accepts a quote

✕ NOT BUILT They don't, in the system. There is no "Accept Quote" button in the inspection report email. There is no customer-facing booking page. Acceptance happens by **phone or reply email**, and the admin then books the job manually.

Archaeology: a table called `client_booking_tokens` exists in the database (created in migration `20251028135212`) — built in anticipation of a tokenised customer booking portal. The endpoint, token issuance, and customer-facing page were never implemented. The table has been dead weight since it was created in migration `20251028135212` .

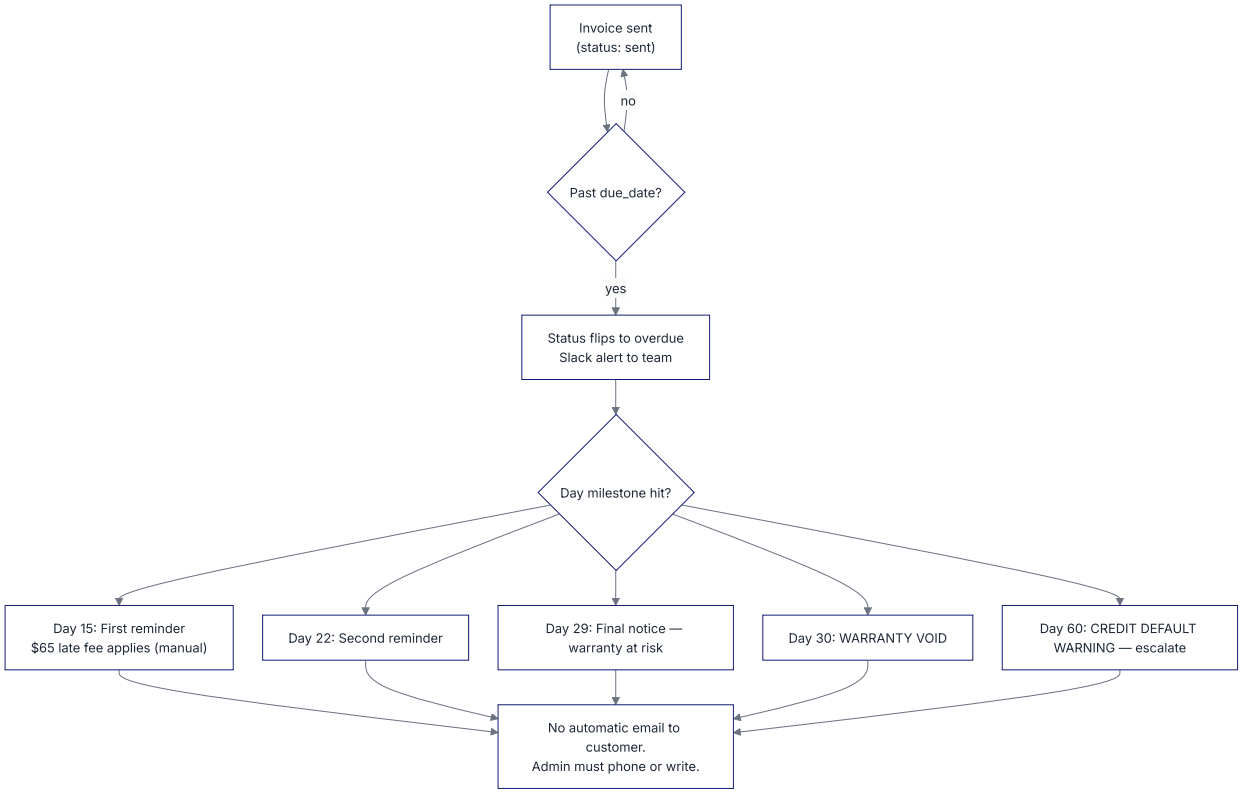
1.5 How the customer pays the invoice

⊙ DELIBERATE **No payment processor integration, by design.** MRC tracks payments as admin-entered data — method, date, and reference — but never processes them. Customers always pay by EFT outside the app. No Stripe, Square, or PayPal is planned; the five payment-method labels in code (`cash` , `visa` , `mastercard` , `bank_transfer` , `cheque`) exist so admin can record *how* a customer paid, not to represent an integration that needs building.

Method (in code)	What actually happens
`bank_transfer`	Customer transfers to MRC's bank account using details from the invoice. No automated reconciliation.
`cash`	Handed over in person.
`visa` / `mastercard`	Card payments are taken via an external terminal, not in-app.
`cheque`	Mailed to MRC.

Once the money lands, an admin opens the customer record, clicks **Mark as Paid**, picks the method from a dropdown, enters the payment date and an optional reference. The invoice status moves to `paid`. The constraint `payment_method IN ('cash','visa','mastercard','bank_transfer','cheque')` is enforced both in the UI dropdown (`src/components/leads/InvoicePaymentCard.tsx` lines 551–557) and at the database level (`supabase/migrations/20260414000004_create_invoices_table.sql` line 32).

1.6 What happens if the customer ignores everything



Overdue handling. The cron runs daily at 23:00 UTC (9 AM AEST). Source: `supabase/functions/check-overdue-invoices/index.ts` lines 21–28.

✓ BUILT Everything in the diagram is real. The milestone messages are pulled directly from the source:

- **Day 15** — "First reminder — $65 late fee applies (admin to charge manually)"
- **Day 22** — "Second reminder — payment still outstanding"
- **Day 29** — "Final notice — warranty at risk"
- **Day 30** — "WARRANTY VOID — 30 days overdue"
- **Day 60** — "CREDIT DEFAULT WARNING — 60+ days overdue, escalate"

> **The customer never receives an automated overdue communication of any kind.** The entire milestone chain — Day 15, 22, 29, 30, 60 — is admin-facing Slack only. There is no overdue email, no SMS, no automated reminder. A human admin reads Slack and decides whether to phone, write, or escalate. If the admin doesn't act, nothing happens. The same statement applies in Section 3.9 from the admin's perspective.

> **Open question for Glen and Clayton.** Should overdue invoices trigger an automated email reminder to the customer? Currently admin decides case-by-case from Slack. An automated first-reminder email at Day 15 (or Day 14 = due date + 0) would reduce admin follow-up load, but could also hurt the retention conversation that happens when a customer rings after a personal chase. Worth discussing before launch.

1.7 Cancelling, disputing, or contacting MRC outside email

DELIBERATE **No customer-facing cancel button, dispute form, or contact page.** When a customer wants to cancel or dispute, they phone or email and speak to a person. This is intentional — a phone call gives the admin a chance to understand what's wrong and potentially save the booking. A self-service cancel button would mean cancellations happen silently, with no opportunity to fix the underlying issue. Customers contact MRC by phoning **1800 954 117** or emailing `admin@mouldandrestoration.com.au`.

The admin-side process — what to do once the call ends — is in Section 5.4.

1.8 The Google review request

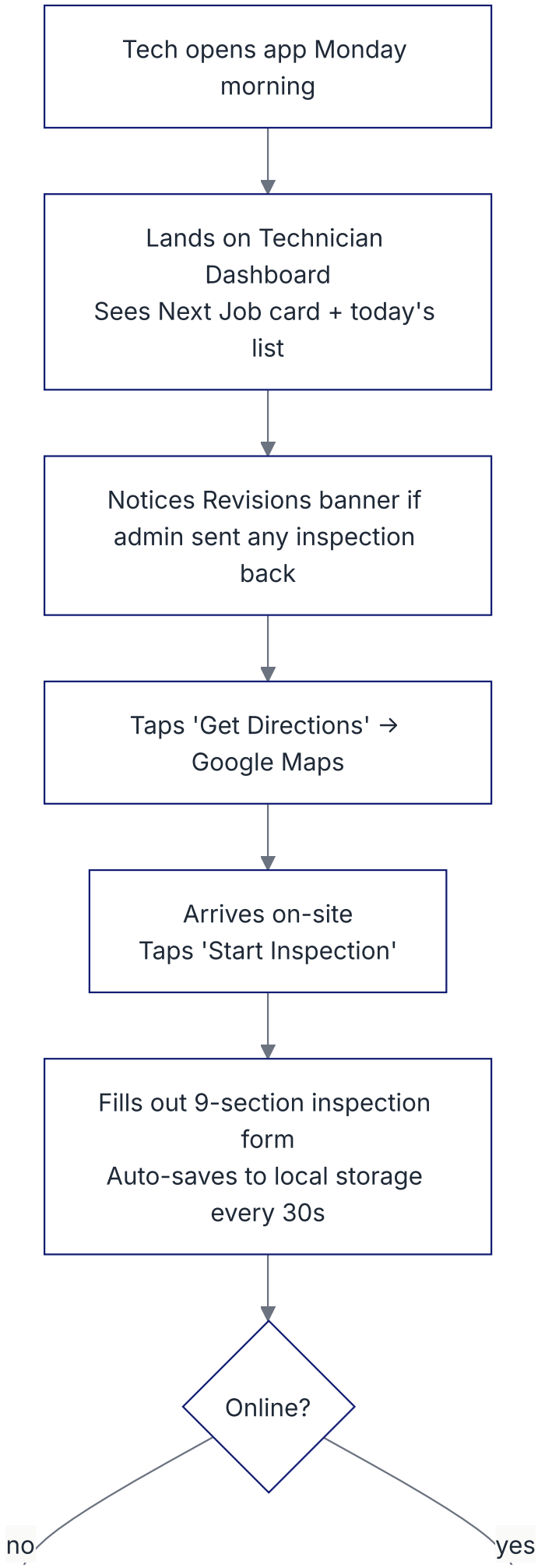
See preview: §1.3 email preview #7 — Google Review Request.

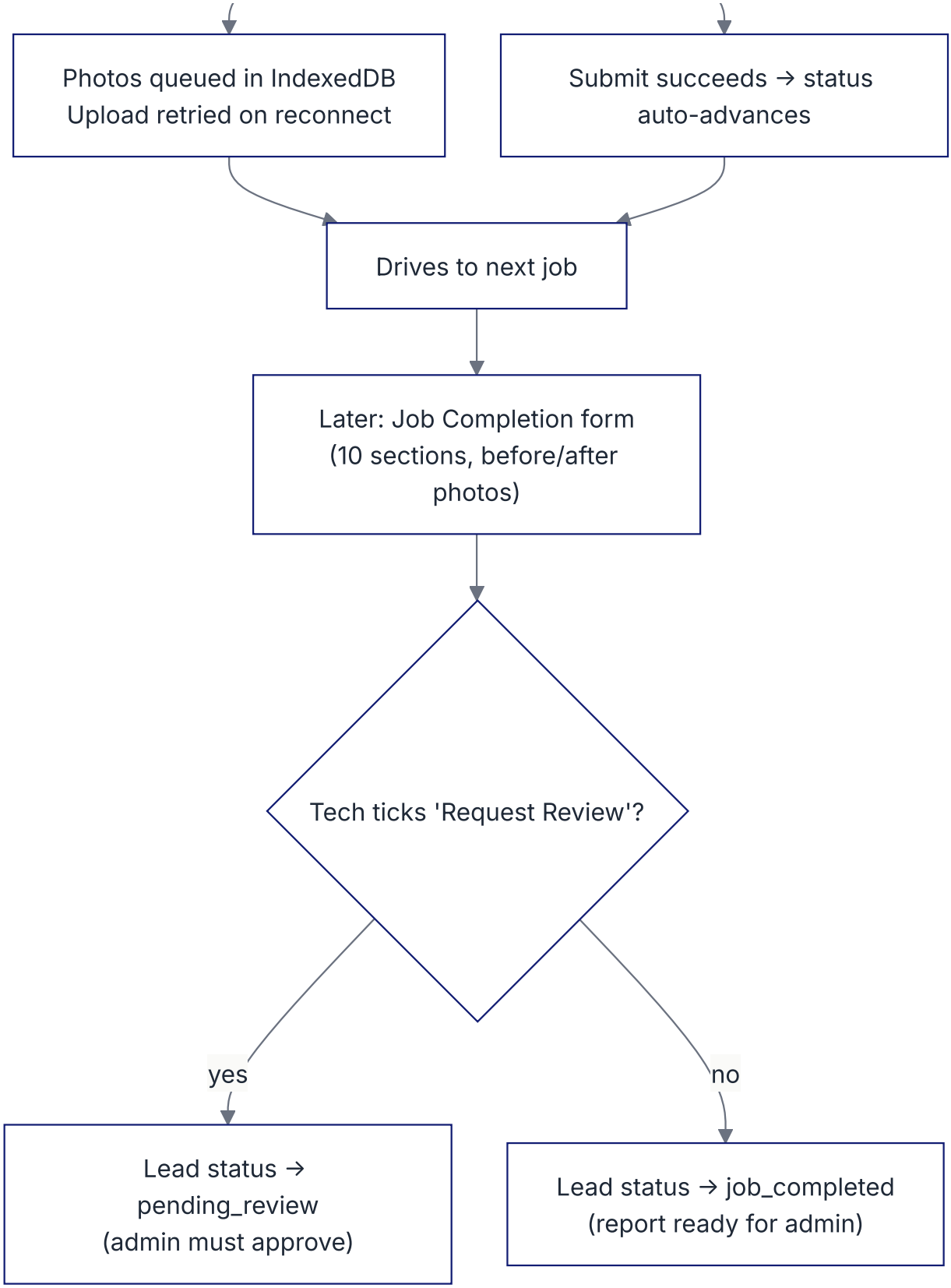
✓ BUILT Manually triggered. When an invoice is marked paid, a "Send Google Review" button appears on the customer record (`src/pages/LeadDetail.tsx:1675`, GoogleReviewSection function). The admin clicks it; the customer receives a "Thank you" email with a single CTA button pointing to `https://g.page/r/CSmcatb7uSq9EBM/review`. The lead status advances to `google_review`.

There is no automated scheduling — if the admin doesn't click the button, the email never sends.

2. Journey 2 — The Field Technician

The technician's week is mobile-first. They open the app on a phone, see today's jobs, drive to a property, fill out a long inspection form (or job-completion form, depending on the visit type), take photos, hit submit, and move on. Below is everything they touch and everything they're blocked from touching.


Tech opens app Monday morning
Lands on Technician Dashboard
Sees Next Job card + today's list
Notices Revisions banner if admin sent any inspection back
Taps 'Get Directions' → Google Maps
Arrives on-site
Taps 'Start Inspection'
Fills out 9-section inspection form
Auto-saves to local storage every 30s
Online?
no
yes



Technician day. The "Request Review" toggle in Section 9 of the job-completion form is the only way a technician escalates a job to admin attention.

2.1 Screens the technician CAN reach

✓ BUILT Eight routes total:

- `/technician` — Dashboard. Today's Next Job card, remaining jobs list, revision alerts.
- `/technician/jobs` — My Jobs (tabs: Today / This Week / This Month / Upcoming / Completed / Revisions).

- `/technician/job/:id` — Customer record (reuses the admin LeadDetail page in role-aware mode).
- `/technician/inspection` — Inspection form.
- `/technician/job-completion/:leadId` — Job Completion form.
- `/technician/alerts` — In-app alerts.
- `/technician/profile` — Profile.
- `/technician/settings`, `/technician/help` — Settings & help.

2.2 Screens the technician CANNOT reach

Enforced by `RoleProtectedRoute` wrappers in `src/App.tsx` and backed by row-level security in the database. If a technician types an admin URL into their browser, they get bounced.

- Admin Dashboard, Lead List, Schedule, Technicians, Reports, Settings.
- The Inspection AI Review queue (`/admin/inspection-ai-review/:leadId`) — admin-only.
- Inspection Report Editor and Job Report Editor — admin-only.
- Other technicians' jobs and customer records. **Why:** the database policy `tech_select_assigned_leads` only returns rows where `assigned_to = auth.uid()` . Even if the UI accidentally rendered another tech's lead, the query would return nothing.
- Email logs, invoices, audit logs — all admin-only.

2.3 How a technician finds out about a new job

⚠ PARTIAL **In-app only.** When the admin assigns a booking, an activity row is inserted; the technician's Alerts page surfaces it on next refresh. The breakdown:

- ⊖ DELIBERATE No SMS — cost and compliance decision (the system has no Twilio / MessageBird wiring).
- ⊖ DELIBERATE No Slack mention to the individual tech — Slack is for the office channel, not per-technician DMs.
- ✕ NOT BUILT No push notification — would require a service worker push subscription. Not yet implemented; sits as a P3 item in Section 11.
- ✕ NOT BUILT No email to the technician on assignment. This is a real gap — sits as a P1 item in Section 11.

> **Open idea — not yet planned.** A technician-only Slack channel (or per-tech DM) could notify Glen or Clayton directly when a new job is assigned to them, a job is sent back for revision, or a booking slot changes. The current setup posts everything to the office channel, which works when the office is watching but doesn't route by technician. Worth discussing before launch — low-effort to wire up if the team wants it.

In practice, the technician learns about new assignments by opening the app — or by the office admin sending a text or calling them outside the system. The same channel-gap pattern applies to send-backs: when admin returns an inspection for revision, the tech sees it on next refresh, no notification fires.

2.4 The Inspection Form — section by section

✓ BUILT Source: `src/pages/TechnicianInspectionForm.tsx`. Validation in `src/lib/schemas/inspectionSchema.ts` lines 13–39. Nine sections, hundreds of fields. Below is the structure with required-vs-optional flags.

Section 1 — Basic Information

- **Job Number** read-only — auto-generated `MRC-YYYY-####` .
- **Triage** optional — textarea for notes about the job.
- **Address** editable — pre-filled from the lead.
- **Inspector** read-only — current user.
- **Requested By** optional — text.
- **Attention To** optional — text.
- **Inspection Date** required — blocks submission if empty.

Section 2 — Property Details

- **Property Occupation** required — Tenanted / Vacant / Owner Occupied / Tenants Vacating.
- **Dwelling Type** required — House / Units / Apartment / Duplex / Townhouse / Commercial / Construction / Industrial.

Section 3 — Areas inspected (repeatable, at least one required)

One block per room. Add and remove with buttons. The form requires at least one area to submit.

- **Area name** required — text (e.g. "Main bedroom").
- **Mould description** optional — textarea.
- **Mould visible locations** optional — multi-select: Ceiling, Cornice, Windows, Walls, Skirting, Flooring, Wardrobe, Cupboard, Contents, Grout/silicone, "No mould visible".
- **Climate readings** — Temperature (°C), Humidity (%), Dew Point (auto-calculated).
- **Moisture Readings** — defaults to 2, can add more. Each has a location label, a percentage, and an optional photo.
- **Room view photos** optional — unlimited.
- **Infrared section** optional toggle — when on: IR photo, natural-light reference photo, and checkboxes (No Active Water Intrusion / Active Infiltration / Past Ingress / Condensation / Missing Insulation).
- **Job time** — labour hours estimated for this area without demolition.
- **Demolition toggle** — when on: demolition time (hours) + description.

Section 4 — Subfloor (optional, toggled on)

- Subfloor required toggle — controls visibility of the rest.
- Observation (textarea), Landscape (Flat Block / Sloping Block).
- Comments (textarea).
- Subfloor moisture readings (repeatable).
- Subfloor photos — up to 20 max.
- Sanitation toggle.
- Treatment time (hours).

Section 5 — Outdoor & Parking

- Temperature (°C), Humidity (%), Dew Point (auto-calculated).
- Outdoor comments (textarea).
- Direction photos (toggle).
- Parking option — Driveway / Street / Permit / Paid / None.

Section 6 — Waste Disposal

- Waste disposal required toggle.
- Size — Small / Medium / Large / Extra Large.

Section 7 — Work Procedure (validation-critical)

- **Treatment methods** at least one required — multi-select. Submit is blocked otherwise (schema line 29).
- **Recommended dehumidifier** — Small / Medium / Large.

Section 8 — Job Summary

- Additional equipment comments (textarea).
- Cause of mould (textarea).
- Additional info for technician (textarea).

Section 9 — Cost Estimate (validation-critical)

- Labour hours: no-demolition, demolition, subfloor.
- Equipment quantities: commercial dehumidifiers, air movers, RCD boxes, equipment days.
- **Manual price override toggle**. When on, the technician enters a manual total inc. GST.
- **Discount %** — capped at 13% by the pricing helper and (separately) by the database constraint on the invoices table.
- Estimated cost (auto-calculated, read-only).

> **Validation rule:** Submit is blocked unless either (a) manual override is on, or (b) at least some labour hours have been entered. See `inspectionSchema.ts:29-37`.

2.5 What blocks submission

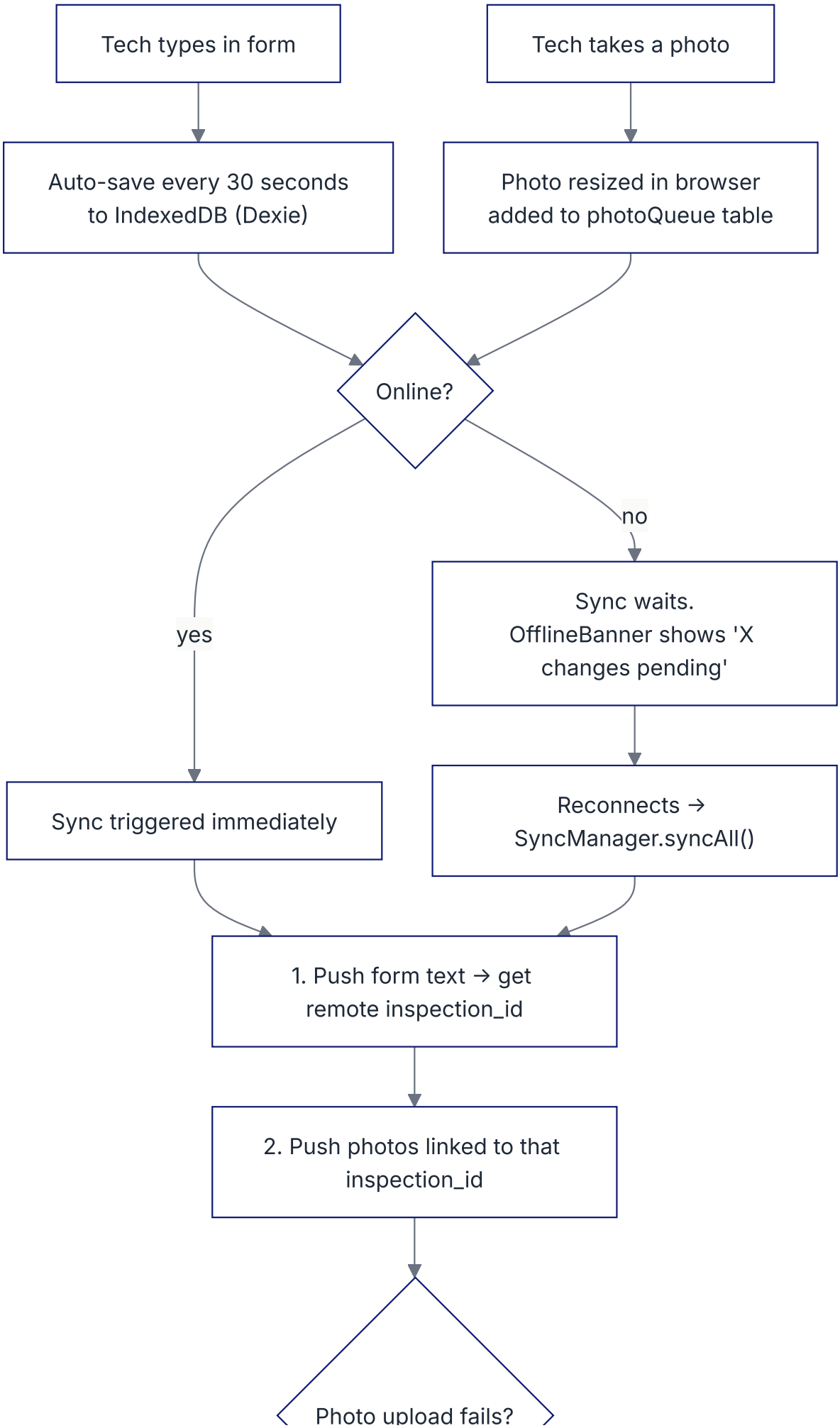
The Zod schema enforces three hard requirements:

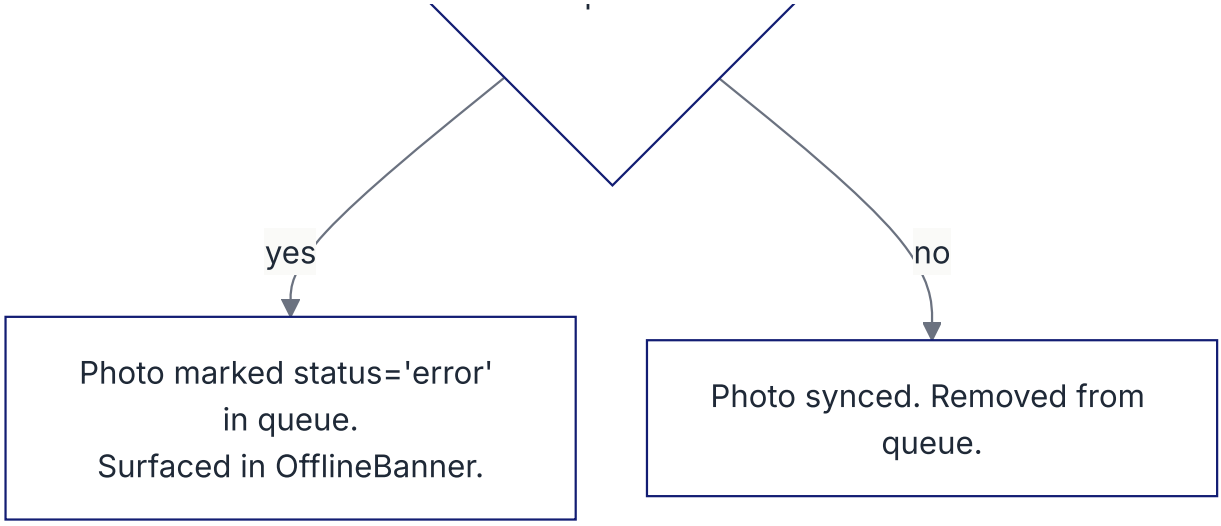
1. **Inspection Date** must be set.
2. At least **one area** must be present.
3. At least **one treatment method** must be selected.
4. At least **some labour hours** OR manual price override.

Photos are not required to submit. Moisture readings are not strictly required. If the technician misses required fields, errors highlight inline and the form scrolls to the first failing field.

2.6 Offline behaviour

✓ BUILT The system is built for spotty signal at job sites.


Tech types in form
Tech takes a photo
Auto-save every 30 seconds to IndexedDB (Dexie)
Photo resized in browser added to photoQueue table
Online?
yes
no
Sync waits. OfflineBanner shows 'X changes pending'
Sync triggered immediately
Reconnects → SyncManager.syncAll()
1. Push form text → get remote inspection_id
2. Push photos linked to that inspection_id
Photo upload fails?



Offline sync. Source: `src/lib/offline/SyncManager.ts`, `src/lib/offline/db.ts`, `src/components/OfflineBanner.tsx`.

- **Auto-save interval** — 30 seconds (constant in `useJobCompletionForm.ts`).
- **What survives offline** — all form text (drafts in IndexedDB `inspectionDrafts` table) and photos (queued in `photoQueue` table after browser-side resize).
- **What the technician sees** — an amber `OfflineBanner` at the top: "You're offline. X change(s) pending." Dismissible with X.
- **What a failed sync looks like** — the photo is marked `status='error'` with the error message. Banner surfaces the count. Retry is implicit on the next online cycle.

> **Caveat:** mutations (writes) in React Query are configured with `retry: false`. So if the tech is briefly online but the request fails for another reason (e.g. RLS rejection, timeout), they get an immediate error toast — no background retry.

2.7 Editing a submitted inspection

⚠ PARTIAL

- **Admin** can always edit, via the route `/admin/inspection/:leadId` which loads the same form component with `adminMode={true}`. On unmount, the form diffs every changed field and writes a single "field edit" entry to the activity log.
- **Technician** editing a previously-submitted inspection is not explicitly blocked at the form level, but is gated by lead status — once the lead has moved past the inspection stage, the form route is no longer reachable from any technician-facing button. There is no formal "lock window" timer.
- The "send back" path: if admin disagrees with a submitted inspection, they can change the lead status back to `inspection_waiting`; the technician then sees a Revisions banner on their dashboard and can re-open the form. (See When Things Go Wrong.)

2.8 The Job Completion Form — 10 sections

✓ BUILT Source: `src/pages/JobCompletionForm.tsx`. Filled out after the remediation work is done. Same offline/auto-save behaviour as the inspection form.

1. **Office Info** — Job number (auto), address snapshot, requested-by, attention-to.
2. **Summary** — SWMS completed toggle required, premises type required, completed-by, completion date required, areas treated at least one.
3. **Before Photos** — selected from the inspection's existing photos; at least one required.
4. **After Photos** — must match the before count; if demolition was done, exactly 4 demolition photos required.
5. **Treatment Methods** — 11 toggles (HEPA vacuuming, surface mould remediation, ULV fogging property, ULV fogging subfloor, subfloor remediation, AFD installation, drying equipment, containment & PRV, material demolition, cavity treatment, debris removal). At least one required.
6. **Chemical Toggles** — 5 (air filtration, water-based, sodium hypochlorite, HEPA vacuumed, sanitised premises).
7. **Equipment Used** — actual quantities × days vs. quoted. Optional but informs invoicing.

8. **Variations** — Scope Changed toggle. When on, "What changed" and "Why changed" become required; "Extra work" and "Scope reduced" optional.
9. **Job Notes** — **Request Review toggle** (the escalation lever). Damages/staining toggles + details. Additional notes.
10. **Office Notes** — admin-only. The component receives `isAdmin={isAdmin}` and hides the section if false.

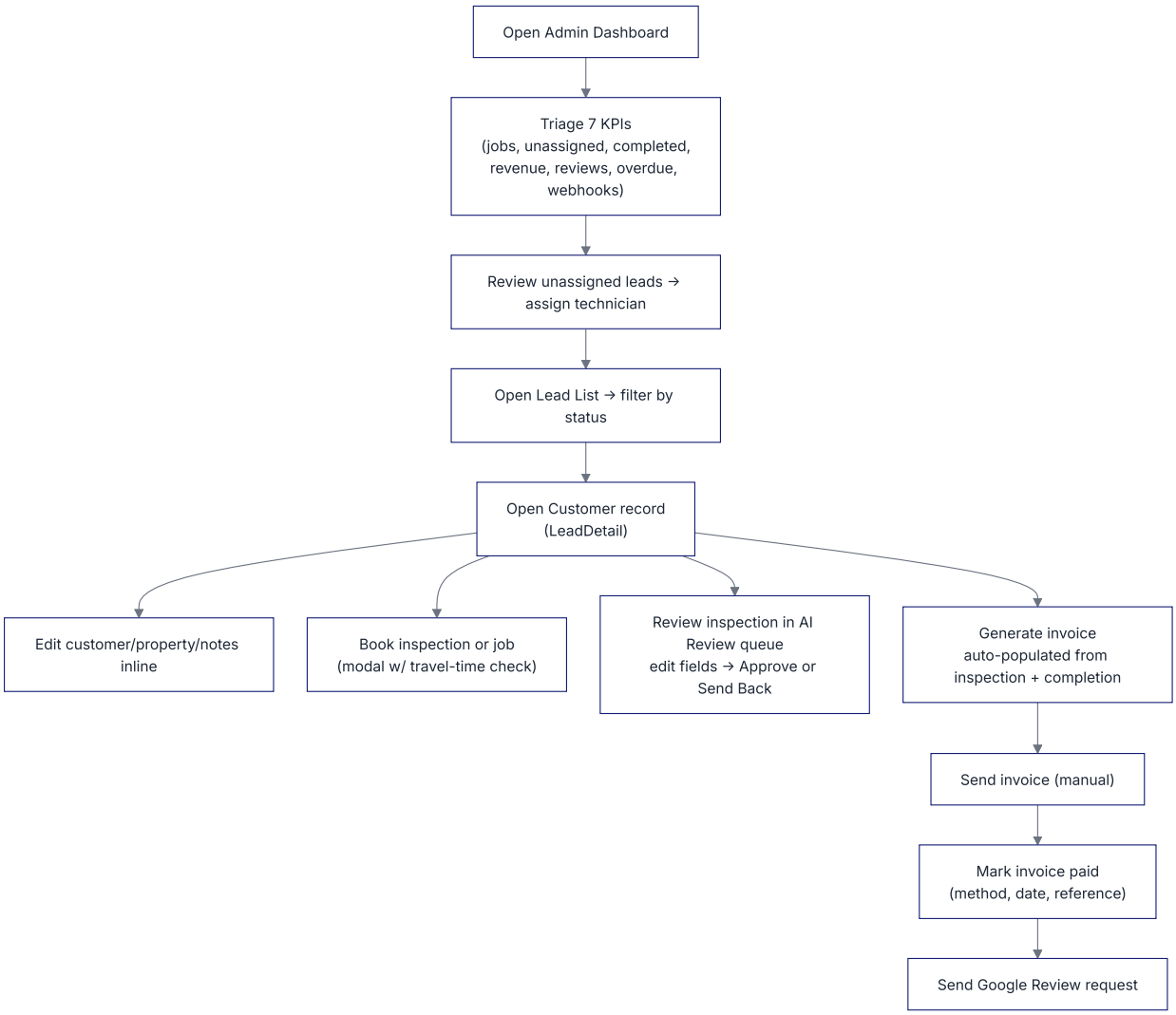
> **What "Request Review" actually does:** `src/lib/api/jobCompletions.ts` line 245 picks the next lead status based on the toggle: `const nextLeadStatus = jc.request_review ? 'pending_review' : 'job_completed'`. That's it. No additional notification fires; the lead just stops in the `pending_review` bucket awaiting admin attention.

2.9 Edge cases the technician hits in real life

Situation	Status	What the system actually offers
Customer not home	✕	No dedicated field or status. Tech writes a free-text note and phones the office.
No access to a particular room	✕	No "skip area" flag. Tech omits the area from the inspection or writes a note in the area's mould-description textarea.
Scope change discovered mid-job	⚠	Section 8 of the job-completion form captures it after the fact (what/why/extra/reduced). No *during-job* reschedule mechanism.
Running late	✕	No reschedule UI for the technician. Admin must reassign manually.
Emergency reassignment	⚠	Admin can reassign a booking to another tech in the schedule view. The reassigned tech sees the update on their next dashboard refresh — no push, no SMS.
Job sent back by admin	✓	The tech's dashboard shows a "Revisions" amber banner with the admin's send-back note and a "Revise Job Completion" button.

3. Journey 3 — The Admin

The admin is the control tower. Almost every status transition is something an admin either triggers directly or rubber-stamps before it advances. They live mostly in two screens: the dashboard for the morning sweep, and the customer record (Lead Detail) for everything else.



Admin paths through the system. The dashboard is the daily entry point; the customer record is where the work actually happens.

3.1 Morning dashboard walkthrough

✓ BUILT Source: `src/pages/AdminDashboard.tsx`, stats hook `src/hooks/useAdminDashboardStats.ts`.

Seven widgets, each a SQL query against Supabase, refreshed via React Query.

Widget	Number shown	Where it comes from
Today's Jobs	Count of inspections scheduled for today (Melbourne TZ)	`inspections` filtered by `inspection_date = today`
Leads to Assign	Count of unassigned active leads	`leads` WHERE `assigned_to IS NULL` AND status in `('new_lead','hipages_lead')` AND not archived
Completed This Week	Count of jobs completed Mon–Sun	`leads` WHERE status in `('job_completed','paid','finished','invoicing_sent')` AND `updated_at ≥ startOfWeek`
Revenue This Week	Sum of `total_inc_gst`	Sum of `inspections.total_inc_gst` for inspections created this week
Pending Reviews	Count of leads waiting for admin review	`leads` WHERE `status='pending_review'` AND not archived
Overdue Invoices	Count + total $ owed	`invoices` WHERE `status='overdue'`; click navigates to filtered Lead List
Failed Webhooks	Count over last 7 days	`webhook_submissions` WHERE `status='failed'` AND created in last 7 days

Below the KPI cards: today's schedule (table on desktop, cards on mobile), recent activity timeline (unified feed of activities + email logs + notifications), an unassigned leads sidebar, the team workload bar chart, and four quick-action buttons (New Lead, Approve Reports, Calendar, Reports).

3.2 The Lead List — every filter, every action

✓ BUILT Source: `src/pages/LeadsManagement.tsx`.

- **Pipeline tabs** — one per status (15+ tabs), with count badges.
- **Search** — searches across name, property address, suburb, email, phone (line 430–439).
- **Sort** — Newest / Oldest / Value (high / low) / Name (line 442–457).
- **View toggle** — Cards (3-column desktop grid) or List (single column).
- **Card content** — name, email, phone, address, lead number, status badge, assigned tech avatar, estimated value, dates.

Per-card quick actions: **View Details**, **Schedule**, **Start Inspection** (techs only), **View PDF**, **Approve**, **Send Email** (opens composer with PDF attachment workflow), **Archive** (soft delete sets `archived_at` — see migration `20260414000002`), **Reactivate**, **Review AI**, **Book Job**, **Not Proceeding** (opens a "remove lead" modal with reason picker — too expensive, went with competitor, no response, etc.).

3.3 Customer Detail (Lead Detail) — every card and button

✓ BUILT Source: `src/pages/LeadDetail.tsx`. The single most-used admin screen; everything about a customer in one scroll.

Components stacked top-to-bottom:

- **Lead header** — name, status badge, assigned technician, key dates.
- **NewLeadView** — editable contact info (name, email, phone).
- **Property card** — editable street/suburb/state/postcode.
- **Issue / Notes** — editable textarea.
- **InspectionDataDisplay** — full breakdown of the inspection form responses (areas, moisture, climate, equipment recommendations, cost calculation).
- **InspectionReportHistory** — list of generated PDF versions; can re-download or re-open.
- **BookInspectionModal / BookJobSheet** — opens via action buttons.
- **JobBookingDetails** — when a job is scheduled, shows technician, date, hours, travel time.
- **JobCompletionSummary** — when the job is completed, shows what was done; admin can open **JobCompletionEditSheet** to edit.
- **InvoiceSummaryCard** — line items, discount, GST, total.

- **InvoicePaymentCard** — generate invoice, edit before sending, mark paid (with payment method dropdown, date, reference), payment timeline showing where the customer is in the overdue milestone schedule.
- **ActivityTimeline** — every event for this customer (status changes, emails sent, bookings, completions, payments, edits).

Action buttons swap based on status: e.g. **Book Inspection** appears on `new_lead`; **Approve Report** on `approve_inspection_report`; **Book Job** on `job_waiting`; **Generate Job Report PDF** on `job_completed`; **Mark Paid** on `invoicing_sent`; **Send Google Review** on `paid`.

3.4 Booking an inspection or job

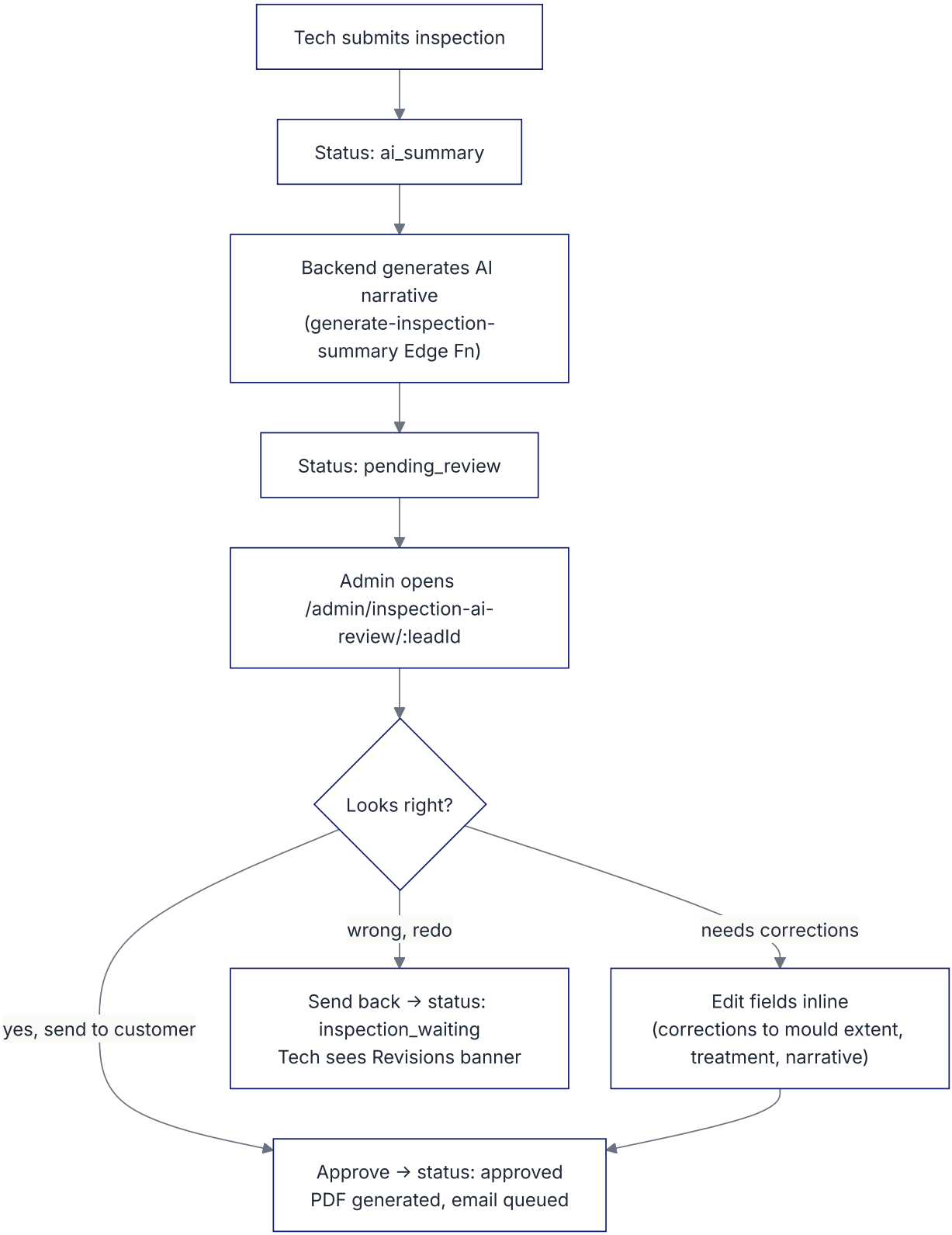
✓ BUILT Source: `src/components/leads/BookInspectionModal.tsx` and `BookJobSheet.tsx`.

The admin opens the modal from a status-driven button. Inputs:

- **Technician dropdown** — fetched via the `manage-users` Edge Function, filtered to technician role.
- **Date picker** — backed by `useBookingValidation` which calls `getRecommendedDates()` with the technician ID, destination address/suburb, and a duration (60 min for inspections, 480 min for jobs).
- **Time slots** — 8 AM to 4 PM in 1-hour increments. Selecting a slot triggers `checkAvailability()` which calls the `calculate-travel-time` Edge Function (Google Maps Distance Matrix) to verify there's enough buffer between the previous job and this one.
- **Manual start address** — admin can override the technician's default starting point to recompute travel time.

On save: `checkBookingConflict()` runs first; if a conflict exists, a toast error fires and the booking aborts. If clear, a row is inserted into `calendar_bookings`, the lead is updated (`status='inspection_waiting'`, `inspection_scheduled_date`, `scheduled_time`, `assigned_to`), an activity log entry is created, and the customer + team are notified (booking confirmation email + Slack post).

3.5 The Inspection AI Review queue



*Inspection review loop. The tech is **not auto-notified** when admin sends back — they see it on their next dashboard refresh as a revisions banner.*

✓ BUILT Source: `src/pages/InspectionAIReview.tsx` for the AI Review queue, and `src/pages/ViewReportPDF.tsx` for the unified report viewer that handles both inspection and job-completion approvals. Admin can edit any field of the inspection inline before approving — corrections are tracked in the activity log via the form's diff-on-unmount mechanism. After approval, the screen transitions to a "Send Report Email" stage (`ViewReportPDF.tsx:2156-2310`) where admin reviews the prepared email, edits the recipient/subject/body if needed, and clicks **Send Email to Customer** to dispatch the PDF as an attachment. The same pattern applies to job-completion reports.

3.6 Quote-setting, the 13% discount cap, GST

✓ BUILT Source: `src/lib/calculations/pricing.ts` lines 14–41.

Tier-based labour rates with linear interpolation:

Labour type	2-hour minimum	8-hour day rate
Non-demolition	$612.00	$1216.99
Demolition	$711.90	$1798.90
Subfloor	$900.00	$2334.69
Construction (future use)	$661.96	$1507.95

Equipment daily rates (never discounted): Dehumidifier $132/day, Air Mover $46/day, RCD $5/day.

Volume discount tiers, applied to *labour only*:

Total labour hours	Discount	Days assumed
0–8	0%	1
9–16	7.5%	2
17–24	10.25%	3
25–32	11.5%	4
33+	13%	5+

> **The 13% cap is enforced in two places by design:**
>
> 1. The pricing helper exports `MAX_DISCOUNT = 0.13`.
> 2. The invoices table has a database-level `CHECK` constraint: `discount_percentage ≥ 0 AND discount_percentage ≤ 13` (`supabase/migrations/20260414000004_create_invoices_table.sql:46`). Even if a future bug bypassed the UI, the database refuses the write.
>
> GST is a flat 10% applied to (discounted labour + un-discounted equipment).

Pricing is entered on Section 9 of the inspection form. The admin can override anything before sending the invoice (line items are JSONB and freely editable). There is **no formal exception-approval workflow** for going above 13% — the database simply refuses.

3.7 Invoice generation

> **Customer invoice email is sent manually, by design.** The **Invoice Summary card** on the lead detail page gives the admin everything they need — line items, totals, GST, payment terms — to paste straight into their own email client and send to the customer. The invoice record is fully managed inside the app; the dispatch step is intentionally manual so the admin can personalise the message and use their preferred email tooling.

✓ BUILT Sources: `src/lib/api/invoices.ts`, `src/components/leads/InvoicePaymentCard.tsx`. The invoice record itself is fully managed inside the app; only the customer-facing email is missing.

- **Invoice number** — auto-generated by a Postgres sequence (`invoice_number_seq`), formatted as `INV-YYYY-####`. Trigger `set_invoice_number` applies on insert. Hardened against race conditions in migration `20260421000001`.
- **Auto-populate** — `buildInvoiceFromLead()` pulls customer details from the lead, line items from the most recent inspection (labour + equipment), actual equipment usage from the most recent job completion, applies the discount.
- **What admin can edit before sending** — every line item, the discount percentage (capped at 13%), the customer email, the property address, the due date (default +14 days), and free-text notes.

- **Sending** — moves status from `draft` → `sent` and records `sent_at`. ⚠ Customer-facing email send for invoices is **not currently wired** in the notifications module — the Slack alert fires, but the customer does not receive an automated invoice email In practice the admin emails the invoice externally.

3.8 Marking an invoice paid

The "Mark as Paid" button opens a dialog with a date picker (defaults today), the payment method dropdown (Bank Transfer / Cash / Visa / Mastercard / Cheque), and an optional payment reference. On submit, the invoice updates (`payment_method`, `payment_date`, `payment_reference`, `status='paid'`, `paid_at`), the lead status moves to `paid`, and a Slack notification fires (fire-and-forget).

3.9 Overdue handling

Already covered in Journey 1, section 1.6. From the admin's perspective: a daily 9 AM AEST cron updates statuses and posts to Slack at the 15 / 22 / 29 / 30 / 60 day milestones. There is **no automated chase email** — the admin reads Slack and phones the customer.

As stated in 1.6, no part of this milestone chain produces an automated customer-facing message. Every escalation past the Slack alert is a manual phone call or email from an admin.

3.10 Reports & Analytics

✓ BUILT Source: `src/pages/Reports.tsx`.

Period filter: Day / Week / Month / Quarter / Year. KPI cards: Total Leads, Conversion Rate, Avg Response Time, Total Revenue. Charts: Timeline (lead volume over time), Status Breakdown (pie), Lead Sources (pie). A "Quick Insights" panel shows top lead source, most common status, and an overall pipeline-health rating (≥30% conversion = healthy, ≥15% = average, <15% = needs attention).

> **Note on Avg Response Time:** the metric is computed dynamically in `src/hooks/useReportsData.ts:177-195`, not hardcoded. It averages the hours between each lead's `created_at` and the earliest `calendar_bookings.created_at` tied to that lead, then rounds to a whole hour. If no leads in the period have any booking yet, the metric returns 0. The proxy (booking creation, not first contact) is good enough today; an earlier draft of this doc described it as a hardcoded "24 hrs" placeholder, which was wrong.

3.11 Admin-only actions and what's logged

- **Soft delete a lead** — sets `archived_at`; queries filter on `archived_at IS NULL`. No hard delete is exposed. (✓)
- **Void invoice** — the API function `voidInvoice()` exists in `src/lib/api/invoices.ts`; status moves to `void`. Whether there's a UI button on every invoice is partially exposed — confirm in the lead detail. (⚠ API present, UI surface limited)
- **Reassign technician mid-job** — open the booking via the schedule or customer record, pick a different technician, save. Conflicts checked. The new tech sees the job on next dashboard refresh; the old tech loses visibility. No automatic notification to either party. (✓)
- **Refunds** — no refund logic, no refund button, nothing. Refunds happen entirely outside the system and would be reflected as a manual adjustment to the invoice. (✗)
- **Audit logging** — split across two migrations. Migration `20260311000001_add_audit_triggers.sql` adds INSERT, UPDATE, and DELETE triggers to four tables: `leads`, `inspections`, `inspection_areas`, `user_roles`. Migration `20260414000004_create_invoices_table.sql:71-75` adds INSERT and UPDATE triggers to `invoices` separately (no DELETE — invoices are voided via status, never hard-deleted). All triggers feed `audit_log_trigger(action_text)`, a SECURITY DEFINER function that writes to `audit_logs`. The audit log itself is tamper-resistant: `prevent_audit_log_delete()` and `prevent_audit_log_update()` triggers raise an exception on any attempt to modify a row. (✓ data captured; ✗ no admin UI for browsing — queryable by developers via Supabase Studio only.)

3.12 Schedule and Technicians screens

- **Schedule** (`src/pages/AdminSchedule.tsx`) — week/day calendar of every booking. Filters by technician and event type. Displays travel time between consecutive jobs. Right sidebar shows unscheduled leads queue, an event details panel for the selected booking, and a cancelled-bookings history.

- **Technicians** (`src/pages/AdminTechnicians.tsx`) — grid of technician cards. Each shows initials avatar (one of 6 rotating colours), name, role badge, weekly inspection count, monthly revenue, upcoming-jobs count, and last sign-in relative time. Stats hook: `useTechnicianStats`.

4. The Notification Map

Every event that fires a notification in the system, mapped to who hears about it and how. Push notifications are **not implemented** anywhere; channels are email, Slack, and the in-app activity feed. The send-email Edge Function rate-limits at **10 emails per minute per IP** (`supabase/functions/send-email/index.ts:11-14`).

Trigger event	Who is notified	How	Example message
Public lead form submitted	Customer + all admins	Email + Slack	"Inspection Request Received — Mould & Restoration Co." Slack: 🏚️ New Lead Received
Admin creates a lead manually	All admins (Slack)	Slack only	"New lead created — ACTION REQUIRED: CALL LEAD"
Inspection booked	Customer + all admins	Email + Slack	"Booking Confirmed" with date/time/tech Slack: Inspection Booked
Inspection reminder (~2 days prior)	Customer	Email (cron)	"Inspection Reminder — 2 days away"
Technician submits inspection form	All admins	Slack + activity feed	"Inspection Report Ready for Approval"
AI summary generated	All admins	Activity feed only	"AI summary generated for inspection"
Admin approves inspection report	Customer + all admins	Email + Slack	"Your Inspection Report is Ready" Slack: Report Approved & Sent
Job booked	Customer + all admins	Email + Slack	"Job Booking Confirmed" with dates/tech/hours
Job completion submitted	All admins	Slack + activity feed	"Job completion submitted by <tech>"
Job report approved & sent	Customer	Email	"Job Completion Report — <jobNumber>" with PDF attached (filename `MRC-JOB-{jobNumber}-Job-Report.pdf`)
Lead status changed (any)	All admins	Slack + activity feed	🔄 Lead Status Changed: old → new
Lead details edited	All admins	Slack + activity feed	"Lead Details Updated — <name>" (which fields changed)
Invoice sent	All admins (Slack)	Slack only	💰 Invoice INV-2026-NNNN sent to <name> — $X *By design — admin sends manually using the Invoice Summary card data.*
Payment received (Mark Paid)	All admins	Slack + activity feed	✅ Payment received for INV-2026-NNNN — $X via <method>
Invoice flips to overdue	All admins	Slack + activity feed	⏰ Invoice INV-2026-NNNN is X day(s) overdue — $X
Overdue milestone (15/22/29/30/60 days)	All admins	Slack + activity feed	🚨 e.g. "First reminder — $65 late fee applies (admin to charge manually)"
Google review request (manual button)	Customer	Email	"Thank You — Would You Leave Us a Review?" with Google link
Tech assigned a new job	Assigned technician	In-app only (next refresh)	Alert appears in `/technician/alerts` — no push, no email, no SMS
Webhook submission failed	Audit only	Database row	Surfaced on Admin Dashboard "Failed Webhooks" widget

4.1 What we don't notify (and why)

The following channels and events are silent — by intention, or by absence. Distinguish carefully:

Deliberately silent

- ⊖ DELIBERATE No SMS to anyone, ever — no Twilio, no MessageBird, no aggregator. Cost and AU compliance considerations.
- ⊖ DELIBERATE No customer-facing push or in-app messaging — customers never log in.

- DELIBERATE No automated overdue communication to the customer — the office wants the human conversation when chasing money. See 1.6.
- DELIBERATE No per-technician Slack DM — Slack is for the office channel; per-tech messaging would fragment the team's shared awareness.
- DELIBERATE No automated customer invoice email. By design, the admin copies the Invoice Summary card data into their own email client and sends the invoice manually. Slack alert fires internally.

Accidentally silent (genuine gaps)

- ✕ NOT BUILT No technician notification (email, push, anything) when a job is assigned or reassigned. Tech learns by opening the app. Section 11, P1.
- ✕ NOT BUILT No technician notification when an inspection is sent back for revision. Same channel gap. Section 11, P1.
- ✕ NOT BUILT No bounce / delivery confirmation loop. The `email_logs` table has columns for bounce status but no Resend webhook listener writes them. Section 11, P1.

5. What Happens When Things Go Wrong

Real failure modes. For each: what the system does on its own, what a human has to do. Where something is missing, it's flagged loudly.

5.1 An email bounces

⚠ PARTIAL The `email_logs` table has status enum values for `bounced`, `soft_bounce`, `spam`, etc. — the *schema is ready*. But there is currently **no Resend webhook listener** implemented to flip those statuses automatically. In practice: the send call succeeds (status `sent`) and the system is unaware if Resend later marks it bounced. An admin would discover the bounce only by:

- Looking at the Resend dashboard directly, or
- Phoning the customer who never replied.

Human required: manually correct the customer's email in the lead and re-send the affected message via the lead detail email composer.

5.2 A customer disputes an invoice

✕ NOT BUILT No formal dispute workflow. The customer phones or emails. The admin can:

- Edit the invoice (line items, total) if status ≠ `paid`.
- Mark it `void` via the API.
- Add a note to the activity log explaining the dispute.

There is no structured dispute reason, no comment thread, and no timer/SLA. The dispute history exists only as free-text activity entries. Risk: tribal knowledge, no consistency between admins.

5.3 Admin disagrees with a submitted inspection

⚠ PARTIAL Two paths:

1. **Edit in place** — admin opens the inspection in adminMode, corrects fields, approves. Changes are diff-logged to the activity timeline.
2. **Send back** — change the lead status to `inspection_waiting`. The technician sees a Revisions banner on their dashboard with the admin's note next time they open the app.

> **Communication gap:** there is no push, email, or SMS to the technician when their work is sent back. The Revisions banner only appears the next time the technician opens the app — often hours later. Admins follow up by phone or text outside the app to close the loop.

5.4 A customer wants to cancel after booking

✕ NOT BUILT No customer-facing cancel button. The flow is:

1. Customer phones MRC.
2. Admin opens the schedule, finds the booking, changes its status.
3. Admin manually emails a cancellation confirmation (or doesn't).
4. Admin manually pings the technician (Slack, phone, text — outside the app).

No structured cancellation reason is captured. The activity log records the status change but not the why.

5.5 Payment doesn't arrive

✓ BUILT (with caveats) The cron flips the invoice to `overdue` the day after the due date. Slack alerts fire on a schedule (Day 15, 22, 29, 30, 60). There is no automated chase email to the customer; admin must phone or write manually. If the customer never pays,

the invoice sits as `overdue` indefinitely. Escalation past 60 days is a manual, outside-the-system process (likely debt collection).

5.6 Photo upload fails mid-sync

✓ BUILT The photo's row in the IndexedDB `photoQueue` table is updated to `status='error'` with the error message. The OfflineBanner displays the count of pending + errored items. The technician can:

- Wait for the next online cycle — sync retries automatically.
- Move to better signal and the sync will pick up.
- If the photo is permanently broken (corrupt, too large, quota exceeded), the only mitigation is to delete it from the queue (limited UI exposure today) and re-shoot.

5.7 The app throws an error at runtime

✓ BUILT Two layers of error boundary:

- **Global ErrorBoundary** — catches anything that escapes the page level. Shows a full-page fallback with "Refresh Page" and "Go to Dashboard" buttons. No technical detail exposed to the user.
- **PageErrorBoundary** — wraps high-risk pages (Inspection AI Review, Inspection Form, Job Completion Form, Admin Inspection Form, Lead Detail). Renders an inline error in just that section with "Try Again" and "Go Back" buttons; the rest of the app keeps working.

Errors are forwarded to **Sentry** with user role + URL context. Breadcrumbs of recent actions are attached. There is no in-app "Report this error" link; users don't know the error has been logged.

5.8 Database connection lost mid-request

React Query is configured with retry: 2 for reads (exponential backoff up to 10 s) and retry: false for writes. So a read failure briefly retries, but a write failure surfaces to the user as an immediate toast error with no background retry. The OfflineBanner appears when `navigator.onLine` goes false. On reconnect, the SyncManager drains the IndexedDB queue.

5.9 Live UI buttons that go nowhere

⚠ MISLEADING UI The "SMS" action button on the Lead Detail page (`src/pages/LeadDetail.tsx:477`) is wired to a handler called `handleSMS()` that has no implementation — clicking does nothing. Visible to admins, no error toast, no console message. Tracked as P1 in Section 11; the fix is either wire it to a Twilio path (which would change the ⊙ DELIBERATE SMS decision) or remove the button. Until then, train new admins to use phone or email and ignore the SMS button.

5.10 No dev environment to test changes against

> **P0 INFRASTRUCTURE GAP.** There is one Supabase project, `ecyivrxjpsmjmexqatym` , and it is production. Every migration, every Edge Function deploy, every schema experiment runs against the live database that customer data sits in. There is no preview, no staging, no rollback rehearsal. Mitigations today: migrations are reviewed manually before deployment; destructive operations require explicit approval per `CLAUDE.md` . Until a dev project is provisioned, every schema change carries the risk of a production-data incident with no rehearsal. Tracked as P0 in Section 11.

6. The Permission Matrix

What each role can and cannot do. Every "yes" or "conditional" is enforced at two layers: the React route guard (`RoleProtectedRoute`) and the database row-level security policy. The database is the source of truth — even if the UI ever leaks a button, the database refuses the operation.

The conditional rule pattern that appears throughout: "only rows where `assigned_to = auth.uid()` for leads, or where the parent lead's `assigned_to = auth.uid()` for child rows like inspections, areas, photos." Translation: **technicians see only their own assigned work, full stop**.

Action	Admin	Technician	Customer (public)
Submit the public lead form	✓	✓	✓
Log into the app	✓	✓	✗
View the Admin Dashboard	✓	✗	✗
View the Technician Dashboard	✗	✓	✗
View any lead in the system	✓	Only if `assigned_to = self`	✗
Edit lead contact info	✓	Limited to `notes` on assigned leads	✗
Edit lead property address	✓	✗	✗
Assign a technician to a lead	✓	✗	✗
Change lead status	✓	✗	✗
View an inspection	✓	Only on assigned leads	✗
Submit an inspection form	✓ (adminMode)	✓	✗
Edit a submitted inspection	✓	Until lead moves past inspection stage	✗
Approve an inspection report	✓	✗	✗
View AI-generated summary	✓	✗	✗
Generate a PDF report	✓	✗	✗
Upload an inspection photo	✓	Only on own inspections	✗
Delete an inspection photo	✓	Only on own inspections	✗
Submit a job completion form	✓ (adminMode)	✓	✗
View the Office Notes (Section 10)	✓	✗ (component hidden)	✗
View the schedule (full week)	✓	Only own bookings	✗
Create / edit / cancel a booking	✓	✗	✗
Generate an invoice	✓	✗	✗
Mark an invoice paid	✓	✗	✗
Void an invoice	⚠ API exposed, UI surface limited (see 3.11)	✗	✗
View email logs	✓	✗	✗
View audit logs	✓ (DB only — no UI)	✗	✗
View all technicians + their stats	✓	✗	✗
Manage user roles	✓	✗	✗
Soft-delete (archive) a lead	✓	✗	✗
Hard-delete anything	✗ (no UI for this)	✗	✗
Edit own profile / change password	✓	✓	✗

Action	Admin	Technician	Customer (public)
Manage trusted devices	✓	✓	✗

Implementation notes:

- **Frontend** — `RoleProtectedRoute` in `src/components/RoleProtectedRoute.tsx` wraps every protected route. Role membership is fetched from the `user_roles` → `roles` tables and cached in the `AuthContext`.
- **Database** — RLS policies enforce the same rules at the row level. Key migrations: `20251111000001_enable_rls_on_leads.sql`, `20251111000002_enable_rls_on_inspections.sql`, `20260414000003_harden_photos_rls.sql`, `20260414000005_harden_leads_inspections_rls.sql`, `20260318000001_fix_remaining_rls_security_issues.sql`.
- **Public form insert** — the policy `allow_public_insert_leads` is intentionally open (anonymous insert allowed), which is what makes the website form work without login. Rate limiting on the receiving Edge Function is the primary abuse mitigation.

7. The Complete Screen Inventory

Every route registered in `src/App.tsx`. **31 routes total** (verified by `grep -c 'path="' src/App.tsx`): four public, twenty-six protected, one catch-all.

Path	Component	Access	Purpose
`/`	Login	Public	Email/password login
`/forgot-password`	ForgotPassword	Public	Request password reset
`/check-email`	CheckEmail	Public	Confirmation that reset email was sent
`/reset-password`	ResetPassword	Public	Set new password (token in URL)
`/admin`	AdminDashboard	Admin	KPIs + today's schedule + activity feed
`/admin/schedule`	AdminSchedule	Admin	Week/day calendar of bookings
`/admin/technicians`	AdminTechnicians	Admin	Roster + per-tech stats
`/admin/technicians/:id`	AdminTechnicianDetail	Admin	Single tech profile + bookings
`/admin/leads`	LeadsManagement	Admin	Pipeline + search + bulk actions
`/admin/inspection-ai-review/:leadId`	InspectionAIReview	Admin	Review + edit AI-drafted inspection content
`/admin/inspection/:leadId`	TechnicianInspectionForm (adminMode)	Admin	Admin edits inspection in same form as tech
`/inspection/:inspectionId/report` `/admin/job-report/:leadId` `/report/:id`	ViewReportPDF†	Admin	Unified PDF viewer; the route detects whether to render an inspection or job report from the URL params
`/admin/reports`	Reports	Admin	Analytics dashboard
`/admin/activity`	Notifications	Admin	System-wide recent activity
`/admin/settings`	Settings	Admin	Admin preferences (shared component)
`/admin/help`	HelpSupport	Admin	Help & documentation (shared)
`/admin/profile`	Profile	Admin	Edit admin profile (shared)
`/lead/new/:id`	NewLeadView	Admin	Initial lead intake view
`/leads/:id`	LeadDetail	Admin	Customer record (most-used screen)
`/technician`	TechnicianDashboard	Technician	Today's jobs + revisions banner
`/technician/jobs`	TechnicianJobs	Technician	My Jobs grouped by date
`/technician/job/:id`	LeadDetail (role-aware)	Technician	Customer record (limited UI)
`/technician/inspection`	TechnicianInspectionForm	Technician	9-section inspection form
`/technician/job-completion/:leadId`	JobCompletionForm	Technician	10-section completion form
`/technician/alerts`	TechnicianAlerts	Technician	In-app alerts feed
`/technician/profile`	Profile	Technician	Edit profile (shared)
`/technician/settings`	Settings	Technician	Tech preferences (shared)
`/technician/help`	HelpSupport	Technician	Help & docs (shared)
`*`	NotFound	All	404 page

Archaeology — files on disk that aren't routed

- i NOTE `src/pages/AdminInvoiceHelper.tsx` — kept on disk; the route was removed when payment tracking was simplified to a card inside LeadDetail. Comment in `src/App.tsx:50` documents the decision. The file is dead code; safe to delete in a cleanup pass.
- i NOTE `src/pages/ViewJobReportPDF.tsx` — previously existed; deleted and unified into `ViewReportPDF.tsx`, which now detects whether to render an inspection report or a job report based on the URL. Comment in `src/App.tsx:51`.

Notes on shared components across roles

- `LeadDetail` — same component for both `/leads/:id` (admin) and `/technician/job/:id` (technician). Conditional rendering hides admin-only buttons when `currentRole !== 'admin'`.
- `Profile`, `Settings`, `HelpSupport` — single implementations reused at admin and technician routes.
- `TechnicianInspectionForm` — used at both `/technician/inspection` (tech, default mode) and `/admin/inspection/:leadId` (admin, with `adminMode` prop).
- `ViewReportPDF` — three URL entry points (consolidated row above): `/inspection/:inspectionId/report` for inspection reports, `/admin/job-report/:leadId` for job reports, and `/report/:id` as the generic viewer used by email-link entry points. The component detects which report type to render from the URL params.

† Three URL entry points, one component. Documented as a single row to avoid implying three independent screens.

8. The Edge Functions DEV

Twelve Supabase Edge Functions live in `supabase/functions/`. Each runs on Deno, deploys via `npx supabase functions deploy <name> --project-ref ecyivrxjpsmjmexqatym`, and reads its secrets from the Supabase project's environment. The table below catalogues every function, what triggers it, what it does, how it behaves on failure, and who calls it.

Name	Trigger	Purpose	Failure mode	Caller
`send-email`	HTTP POST	Sends a transactional email via Resend. Validates payload with Zod, logs to `email_logs`, retries 3× with exponential backoff (1 s → 2 s → 4 s), skips retry on 4xx except 429.	Rate-limited at 10 requests/min per IP (`send-email/index.ts:13-14`); fire-and-forget from caller (errors swallowed at caller).	Frontend admin email composer (Inspection Report Editor at `ViewReportPDF.tsx`; Lead List composer at `LeadsManagement.tsx`); other Edge Functions (`receive-framer-lead` calls Resend directly, not via `send-email`; `send-inspection-reminder` uses this dispatcher).
`send-inspection-reminder`	pg_cron `0 * * * *` (hourly)	Queries upcoming inspections and emails reminders ~2 days before each booking; updates `calendar_bookings.reminder_sent` to true.	Per-email failures logged but do not block the cron run; whole run errors surface only in Supabase function logs.	Cron only.
`send-slack-notification`	HTTP POST	Posts a Slack Block Kit message to the configured webhook. Used for every status change, booking, payment, and overdue milestone.	Fire-and-forget; failures land in function logs only. In-memory rate limit 10/min per IP.	Frontend on every status change; other Edge Functions (`check-overdue-invoices`, `receive-framer-lead`).
`generate-inspection-pdf`	HTTP POST	Fetches an inspection + areas + photos, fills the HTML template stored in the `pdf-templates` bucket, returns a populated HTML URL or raw HTML for client-side PDF rendering.	Returns 404 if inspection not found. Service-role read bypasses RLS.	Admin UI on the Inspection Report Editor.
`generate-job-report-pdf`	HTTP POST	Same pattern as the inspection PDF, for job-completion reports. Downloads photos, embeds as base64 (no signed-URL TTL), populates the job-report HTML template.	Returns 404 if the job completion record is not found. Template `pdf-templates/job-report-template.html` is confirmed present in storage (verified by working production sends).	Admin UI when an admin opens or regenerates a job report.
`generate-inspection-summary`	HTTP POST	Calls OpenRouter (Gemini family) to draft AI inspection narrative — "What we found", "Treatment plan", "Detailed analysis". Three-model fallback chain.	If all three models fail, returns 500 with the last error message; admin sees an error toast and can retry.	Admin UI on the AI Review page.
`calculate-travel-time`	HTTP POST	Calls Google Maps Distance Matrix API for technician-to-property travel time. Used for booking conflict checks and recommended-date suggestions.	If Google Maps is unreachable, the booking modal shows a warning and the admin can override; the database also has a fallback `calculate_travel_time()` SQL function backed by the `suburb_zones` 4×4 zone matrix.	Admin UI from `BookInspectionModal.tsx` and `BookJobSheet.tsx`.
`receive-framer-lead`	HTTP POST (webhook from Framer)	Public webhook receiver. Smart-extracts name/email/phone/address from arbitrary form payloads using regex heuristics, creates a lead, sends a confirmation email, posts a Slack alert. Logs to `webhook_submissions`.	Rate-limited 5 submissions/hour per IP. On parse failure, still inserts a `webhook_submissions` row with status `failed` for admin review (surfaces on the dashboard "Failed Webhooks" widget).	External — the public Framer site posts to this endpoint.
`manage-users`	HTTP POST	Admin CRUD on users + role assignment via Supabase Auth admin API. Wraps service-role calls.	Returns standard 4xx for permission/validation errors.	Admin UI from the Technicians page and lead-assignment dropdowns.
`seed-admin`	HTTP POST	One-shot setup function that creates the initial admin account. Not used post-launch.	Returns 409 if an admin already exists.	Manual call by a developer during initial provisioning. **i NOTE** safe to delete in a future cleanup pass.

Name	Trigger	Purpose	Failure mode	Caller
`export-inspection-context`	HTTP POST	Exports a complete inspection record (lead + areas + photos + readings) as JSON. Used for AI prompt refinement and debugging.	Returns 404 if inspection not found.	Admin tooling, infrequently. Not exposed in primary UI.
`check-overdue-invoices`	pg_cron `0 23 * * *` (daily 23:00 UTC = 9 AM AEST)	Marks invoices overdue once `due_date < today`, posts a Slack alert with milestone text on Days 15, 22, 29, 30, 60. Inserts an activity row per change.	Per-invoice errors collected and reported in the run summary; the cron itself swallows them so one bad row doesn't poison the whole run.	Cron only.

Environment variables consumed across the 12 functions: `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY` (universal), `RESEND_API_KEY` (send-email), `SLACK_WEBHOOK_URL` (send-slack-notification, check-overdue-invoices, receive-framer-lead), `OPENROUTER_API_KEY` (generate-inspection-summary), `GOOGLE_MAPS_API_KEY` (calculate-travel-time). All set via Supabase project secrets, not committed to the repo.

9. The Database Shape DEV

One Supabase Postgres project (`ecyivrxjpsmjmexqatym`) holds everything. RLS is enabled on every business table; the canonical pattern is "admin sees all rows; tech sees only rows tied to a lead where `assigned_to = auth.uid()` ". This section inventories the live tables, the SQL helpers and triggers that enforce business rules, the storage buckets, and the realtime subscriptions.

9.1 Tables

Active business tables first, then system / supporting tables, then dead tables that exist in the schema but are not referenced by any code in `src/` . **Scale** is qualitative: *tiny* (≤10 rows), *small* (tens), *growing* (unbounded operational data). **RLS** shows whether policies are present and the dominant rule.

Table	Purpose	Scale	RLS	Migration	Notes
`leads`	Lead pipeline records — contact, property, status, assignment	Growing	Admin all; tech only assigned	20251028135212	Soft-delete via `archived_at` (20260414000002). Public INSERT allowed for the website form.
`inspections`	Site visit records — findings, climate, pricing, AI summary	Growing	Admin all; tech via parent lead	20251028135212	Largest schema (~95 columns). Hardened in 20260414000005.
`inspection_areas`	Per-room observations within an inspection	Growing	Inherits via inspection	20251028135212	Audit triggers in 20260311000001.
`moisture_readings`	Per-area moisture % readings	Growing	Inherits via area	20251028135212	—
`subfloor_data`	Optional subfloor inspection block	Growing	Inherits via inspection	20251028135212	1:1 with inspection. FK fix in 20251120000001.
`subfloor_readings`	Per-subfloor moisture readings	Growing	Inherits via subfloor	20251028135212	—
`photos`	All field photos (inspection + job completion)	Growing	Tech can act on own; admin all (20260414000003)	20251028135212	`photo_category` column added later for before/after/demolition tagging.
`calendar_bookings`	Scheduled appointments — inspection or job	Growing	Admin all; tech own bookings	renamed in 20251111000016 (was `calendar_events`)	Realtime enabled in 20260209100000. A handful of older comments and type definitions still reference the pre-rename name.
`job_completions`	Phase 2 — what was actually done on remediation	Growing	Admin all; tech own	created via Studio (no migration in repo)	FK target of `invoices.job_completion_id`. Code path in `src/lib/api/jobCompletions.ts`.
`invoices`	Billing — line items (JSONB), discount cap, payment tracking	Growing	Admin only	20260414000004	13% discount CHECK, payment_method CHECK with 5 values, INSERT/UPDATE audit triggers (no DELETE).
`profiles`	Lightweight user extension (display name, onboarding flag)	Small	Authenticated read	20260209000000	Real auth lives in `auth.users`.
`user_roles`	Role grants per user	Small	Admin manages	20251028135212	Audit triggers in 20260311000001.

Table	Purpose	Scale	RLS	Migration	Notes
`roles`	Role definitions	Tiny	Public read	20251028135212	Three rows: admin, technician, manager (manager unused).
`activities`	Human-readable per-lead event timeline	Growing	Inherits via lead	20251028135212	Populated by `trigger_lead_activity()` on lead INSERT/UPDATE (20251112000020).
`audit_logs`	Immutable change history (old/new values)	Growing	Admin only; UPDATE/DELETE blocked by triggers	20260308000001	Tamper-resistant by design. No admin UI for browsing.
`error_logs`	Application-side errors	Growing	Restricted to `auth.uid() = user_id` (20260318000001)	20260308000001	Sentry is the primary error sink; this table is a secondary record.
`email_logs`	Outbound email delivery records	Growing	Admin only; service role write (20251111000008)	20251111000008	Status enum includes `bounced` / `soft_bounce` / `spam` but no Resend webhook listener writes those — Section 5.1.
`notifications`	In-app notification bell	Growing	Per-user	20251029103512	—
`login_activity`	Login attempts (success/fail, device, location)	Growing	Anon insert only for failed (20260318000001)	20260125000001	—
`user_devices`	Trusted device fingerprints	Small	Per-user	20260125000001	—
`user_sessions`	Active session tracking	Growing	Admin can force-logout	20260125000001	—
`suspicious_activity`	Auto-flagged security events	Small	Admin only; insert restricted (20260318000001)	20260125000001	—
`webhook_submissions`	Audit trail for incoming webhooks (Framer)	Growing	Public INSERT; admin read	20260421000000	Feeds the "Failed Webhooks" KPI on the admin dashboard. `retry_count` column tracks reprocessing attempts.
`suburb_zones`	4-tier zone matrix for travel-time calc	Small	Public read	20251111000004 (seeded 20251111000005)	Backs the SQL `calculate_travel_time()` fallback when Google Maps is unavailable.
`app_settings`	Generic key-value config	Tiny	Hardened in 20260405000001	20251111000012	Used by inspection-number sequence; otherwise minimal.
Dead tables — exist in the schema, no references in `src/`					
`sms_logs`	SMS audit trail	Empty	—	20251111000009	SMS is DELIBERATE not built. Drop candidate.
`offline_queue`	Server-side offline queue	Empty	—	20251111000010	Superseded by client-side Dexie. Drop candidate.

Table	Purpose	Scale	RLS	Migration	Notes
`pdf_versions`	Generated PDF version snapshots	Empty	—	20241221000000	Table created; no INSERT statements anywhere. Drop or wire up.
`editable_fields`	Inline-editable field metadata	Empty	—	20241221000000	No CRUD anywhere. Drop candidate.
`equipment`, `operating_hours`, `pricing_settings`	Config-table scaffolding	Empty	—	20251028135212	Created in the early schema, never wired. Drop candidates.
`password_reset_tokens`, `booking_tokens`, `client_booking_tokens`	Legacy token tables	Empty	—	20251028135212, 20251029103512	Replaced by Supabase Auth (password reset) and never built (booking tokens). Drop candidates.
`company_settings`	Company-wide settings	Empty	—	20251028133857	Created, never written. Drop candidate.

The generated TypeScript types in `src/integrations/supabase/types.ts` are stale: they predate the `invoices`, `job_completions` and `webhook_submissions` tables. The application typechecks against the runtime shape (errors only show at integration boundaries) but a regenerate is overdue — tracked in Section 11.

9.2 Functions and triggers

The following SQL functions and triggers enforce business rules at the database layer. They are the reason certain bugs are impossible: even if the UI is bypassed, the database refuses.

- `is_admin()` and `is_admin(uuid)` — RLS helper used by every admin policy. Defined in `20260109000003_fix_is_admin_function.sql` (replaced an earlier version that queried a since-dropped `users` table).
- `generate_invoice_number()` + trigger `set_invoice_number` — auto-formats `INV-YYYY-NNNN` on INSERT. Sequence `invoice_number_seq` backs the counter. Hardened in `20260421000001_harden_generate_invoice_number.sql` with an explicit `search_path`.
- `generate_inspection_number()` — formats `INS-YYYYMMDD-XXX` with a daily reset (20251111000011, race-fix in 20251111000012).
- `generate_lead_number()` + trigger `add_lead_number` — auto-numbers leads (20251112000003).
- `trigger_lead_activity()` + trigger `add_lead_activity` — writes a row to `activities` on every lead INSERT/UPDATE (20251112000020). This is why the activity timeline on Lead Detail is always in sync.
- `check_booking_conflicts()` and `has_travel_time_conflict()` — used by the booking modal before INSERT to prevent double-booking and to enforce the 15-minute buffer plus calculated travel time between consecutive jobs (20251111000011).
- `calculate_travel_time()` + `get_zone_by_suburb()` — the SQL fallback for the Google Maps Distance Matrix call, backed by the `suburb_zones` 4×4 matrix (20251111000006).
- `audit_log_trigger(action_text)` — SECURITY DEFINER function that writes old/new values to `audit_logs`. Triggered on `leads`, `inspections`, `inspection_areas`, `user_roles` (all of INSERT/UPDATE/DELETE, migration 20260311000001) and on `invoices` (INSERT/UPDATE only, migration 20260414000004).
- `prevent_audit_log_delete()` + `prevent_audit_log_update()` — raise an exception on any attempt to modify `audit_logs` rows. The audit log is append-only by enforcement, not convention.
- `update_updated_at_column()` — generic touch trigger on most tables.

9.3 Storage buckets

- `inspection-photos` — every field photo (inspection or job completion). Filename pattern is `<lead_id>/<photo_id>.jpg`. RLS scoped so techs can read only their assigned leads' photos.
- `pdf-templates` — HTML template files used by the two PDF generators. Both templates confirmed present: `inspection-report-template.html` and `job-report-template.html` (the latter verified by working production sends).
- `profile-photos` — user avatars (added in `20260415000000_profile_photos_bucket.sql`). Public read for display. Per-user write via RLS scoped to user folder.

9.4 Realtime subscriptions

Three frontend hooks subscribe to Supabase realtime channels. This is what keeps the schedule and dashboards live without polling:

- `src/hooks/useInspectionLeads.ts` — subscribes to `inspections` changes.
- `src/hooks/useTechnicianJobs.ts` — subscribes to `calendar_bookings` changes (realtime explicitly enabled in `20260209100000_enable_realtime_calendar_bookings.sql`).
- `src/hooks/useNotifications.ts` — subscribes to `activities` updates so the activity feed refreshes live.

9.5 PWA / Service Worker

`vite.config.ts` registers `vite-plugin-pwa`. Workbox runtime caching: Google Fonts CacheFirst (1 year); Supabase REST NetworkFirst (1 hour); Supabase Auth, Storage, and Functions are NetworkOnly (never cached). Manifest: `name="MRC Field"`, `start_url="/technician"`, maskable icons at 192×192 and 512×512. The service worker is what makes the app installable as a home-screen app and what supports the offline path documented in Section 2.6.

10. The Integrations

Eight external services touch the system. For each: what it's used for, where the credentials live, what breaks if the service goes down, and who notices.

10.1 Supabase

The platform layer. Provides the Postgres database, authentication, file storage, Edge Functions runtime, and realtime subscriptions. One project: `ecyivrxjpsmjmexqatym`. Credentials: `SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY` in Edge Function secrets; `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` in Vercel frontend environment. **If Supabase goes down:** the entire app is dark. Login fails, every query fails, the offline banner appears for online users. Technicians on the road can still fill out forms (data queues to IndexedDB) but sync is blocked until Supabase recovers. Sentry catches the failures; the team notices via the activity dashboards going stale.

10.2 Resend

Outbound transactional email. All branded customer emails — booking confirmations, report deliveries, Google review requests — go through Resend via the `send-email` Edge Function. Sender domain is `mrcsystem.com` as of commit `cf2b3d8`; the planned switch to `mouldandrestoration.com.au` is tracked in Section 11. Credentials: `RESEND_API_KEY` in Edge Function secrets. **If Resend goes down:** customer emails fail silently (caller does fire-and-forget). The `email_logs` table records the failure but no admin is paged. Technicians and admins notice only when a customer complains about not receiving a report. Bounces are a separate failure mode: see Section 5.1.

10.3 Slack

Office team notifications. Every status change, every booking, every payment, every overdue milestone posts to a single configured webhook channel via the `send-slack-notification` Edge Function (Block Kit format). Credentials: `SLACK_WEBHOOK_URL` in Edge Function secrets. **If Slack goes down or the webhook URL becomes invalid:** Slack posts fail silently. The team loses situational awareness — new leads don't pop up in chat, payments aren't visible — and will only notice when someone checks the admin dashboard expecting recent activity. The system itself continues to operate normally; only the team's nervous-system feed is interrupted.

10.4 Google Maps

Distance Matrix API for travel-time calculation between technician locations and job sites. Used by `calculate-travel-time` Edge Function and consumed by the booking modals to suggest available dates and detect travel-buffer conflicts. Credentials: `GOOGLE_MAPS_API_KEY` in Edge Function secrets. **If Google Maps is unreachable:** the booking modal shows a warning and the admin can override; the database has a fallback `calculate_travel_time()` SQL function backed by the `suburb_zones` 4×4 zone matrix (CBD / Inner / Middle / Outer). Bookings still work; travel-time accuracy degrades. The admin notices through inaccurate travel estimates on the schedule view.

10.5 OpenRouter (Gemini)

AI inspection summary generation. Called by `generate-inspection-summary` with a 3-model fallback chain (primary, then two backups) so a single model going offline doesn't break the feature. Credentials: `OPENROUTER_API_KEY` in Edge Function secrets. **If OpenRouter is completely down or all three models fail:** the AI Review screen shows an error; the admin can retry or write the summary by hand. The lead doesn't progress until the summary exists. The admin notices immediately.

10.6 Sentry

Application-side error tracking and performance monitoring. Wraps the React tree (`src/components/ErrorBoundary.tsx`, `PageErrorBoundary`), captures uncaught exceptions with breadcrumbs and user role context, and attaches replay snapshots. Configured via `VITE_SENTRY_DSN` at runtime; source-map upload uses build-time variables. **If Sentry is down:** the app continues working; the team simply loses the error feed. Errors still bubble up to the user via error boundaries. New issues won't be visible in the Sentry dashboard until it recovers.

10.7 Vercel

Hosting and CDN. Auto-deploys the React app from the `main` branch to preview URLs and from the `production` branch to the live URL. Enforces security headers (CSP, X-Frame-Options, Referrer-Policy). Frontend environment variables (`VITE_*`) are managed in the Vercel project settings. **If Vercel goes down:** the app is unreachable for everyone. Edge Functions and the database remain online (separate Supabase infrastructure) but no UI can reach them. The team notices instantly — nobody can log in.

10.8 Framer

The public marketing/booking website. Submits new leads to the `receive-framer-lead` Edge Function as webhook POST. The Edge Function smart-extracts fields and creates a lead. Credentials: none on the MRC side — Framer calls a public endpoint (rate-limited 5/hr/IP). **If Framer goes down:** the website is offline; nobody can submit a new request. Existing leads, the admin app, and the technician app are unaffected. Marketing notices first.

10.9 hipages — not an integration

`hipages_lead` appears in the lead status enum, suggesting integration. It is not. There is no API call, no webhook handler, no scheduled poll. "Hipages" is a manual flag an admin sets when a lead originated from hipages.com.au. Documented here because new readers reasonably assume otherwise.

11. Known Issues & Backlog

A candid list of what is broken, missing, or technical debt — prioritised by operational impact. P0 items affect revenue or safety and should be addressed before the next handover. P1 items create friction operators must work around. P2 items are tech debt worth cleaning. P3 items are nice-to-haves or DELIBERATE business decisions documented for the record.

P0 Revenue-blocking or safety-critical

- **No development Supabase project** — every migration, every Edge Function deploy, every schema experiment runs against the production database. Operational impact: any mistake risks customer data; there is no rollback rehearsal. *Fix:* provision a second Supabase project, run all migrations against it, and point Vercel preview deployments at it via Preview environment variables.

P1 Operational friction

- **SMS button on Lead Detail is wired to an empty stub** — `src/pages/LeadDetail.tsx:477` defines `handleSMS()` but the function body is empty. The button is visible to admins; clicking does nothing. Operational impact: misleading UI; new admins waste time wondering why "SMS" doesn't work. *Fix:* either implement the SMS path (which would change the DELIBERATE SMS decision and require Twilio integration), or remove the button. Recommend remove.
- **Email sender domain is `mrcsystem.com`, not `mouldandrestoration.com.au`** — customers see mixed branding (website is `.au`, emails come from `.com`). Listed in `docs/TODO.md` as a weekend task. *Fix:* configure DNS (SPF, DKIM, DMARC) for the .au domain in Resend; flip the default `from` in `supabase/functions/send-email/index.ts:185`.
- **Resend bounce webhook listener is not built** — the `email_logs` status enum has values for `bounced`, `soft_bounce`, and `spam` but nothing writes them. Operational impact: the team is unaware when an email never reached the customer until the customer calls in. *Fix:* add an Edge Function that receives Resend webhook events and updates `email_logs` by `provider_message_id`.
- **Technician is never push-notified for any lifecycle event** — same channel-gap pattern in two visible places:
  - *Job assignment* — when admin assigns or reassigns a booking, the tech learns by opening the app. See Section 2.3.
  - *Send-back* — when admin returns an inspection for revision, the tech sees a Revisions banner only on next refresh. Rework can sit unaddressed for hours.

  Operational impact: assignments and reworks rely on the office texting or phoning the tech outside the app. *Fix:* a single Edge Function that emails the technician on relevant `activities` table inserts (assignment, reassignment, status revert). One ship, both gaps closed.
- **Audit log has no admin UI** — `audit_logs` is populated faithfully but only queryable through Supabase Studio. Operational impact: an admin investigating "who changed this?" has to find a developer. *Fix:* a simple read-only audit log viewer page filtered by lead or by user.

P2 Tech debt

- **Orphan file:** `src/pages/AdminInvoiceHelper.tsx` exists on disk but has no route. The route was removed when payment tracking was simplified into a card on Lead Detail. *Fix:* delete the file.
- **Dead tables to drop:** `sms_logs`, `offline_queue`, `pdf_versions`, `editable_fields`, `equipment`, `operating_hours`, `pricing_settings`, `password_reset_tokens`, `booking_tokens`, `client_booking_tokens`, `company_settings`. None have any `from('<name>')` caller in `src/`. See Section 9.1 for context. *Ship-ready migration:*

```
-- supabase/migrations/YYYYMMDDHHMMSS_drop_dead_tables.sql
DROP TABLE IF EXISTS
  sms_logs,
  offline_queue,
  pdf_versions,
  editable_fields,
  equipment,
  operating_hours,
  pricing_settings,
  password_reset_tokens,
  booking_tokens,
  client_booking_tokens,
```

```
  company_settings
CASCADE;
```

- **Stale generated types:** `src/integrations/supabase/types.ts` predates the `invoices`, `job_completions`, and `webhook_submissions` tables. *Fix:* regenerate via `npx supabase gen types typescript --project-id ecyivrxjpsmjmexqatym`.
- **Naming inconsistency:** `inspections.inspector_id` versus `calendar_bookings.technician_id` — same concept, two names. *Fix:* rename column in a future migration; risky because of view dependencies, so deferring is acceptable.
- **Photo sync queue retry behaviour is undocumented** — visible to developers via `src/lib/offline/SyncManager.ts` but the failure / retry / give-up boundary isn't explicit. *Fix:* add inline JSDoc and surface a tech-facing UI for stuck items.
- **Customer-preferred date and time written to `inspection_scheduled_date` as if confirmed.** The Framer form captures the customer's preferred date and time. The Edge Function (`receive-framer-lead/index.ts:616-617`) writes these into `inspection_scheduled_date` and `scheduled_time` on the leads table — the same columns used for real, admin-confirmed bookings. Operational impact: the Admin Dashboard "Today's Jobs" KPI and the Schedule view surface customer preferences as if they were real appointments; reports that filter by inspection date include preference data; there is no way to distinguish "customer wishes Monday 2 pm" from "technician actually booked Monday 2 pm". *Fix:* add `customer_preferred_date` and `customer_preferred_time` columns to the leads table; update `receive-framer-lead` to write there instead; leave `inspection_scheduled_date` null until admin books formally via `BookInspectionModal`; admin UI surfaces the preference in the Book Inspection flow as a starting suggestion, not a real slot.

P3 Nice-to-have

Real but non-urgent gaps. These are work to do eventually, not decisions already made.

- **Push notifications for technician assignments** — would require a service worker push subscription and Firebase or equivalent. Not blocking; the technician-notification email fix (P1 above) closes the operational gap with less infrastructure.
- **Realtime collaboration on inspection edit** — if two admins open the same inspection at once, last-write-wins. Not yet a real conflict at current user counts.

DELIBERATE Out of scope by decision

Listed for the record so they don't reappear as backlog items. Each is a business decision that can be revisited — but until revisited, treat them as designed-out, not missing.

- **Customer self-service portal** (cancel / dispute / view invoice) — Not yet planned. Today's volume doesn't justify it; the retention conversation when a customer phones is the more valuable business signal.
- **Stripe / payment processor integration** — current invoice volume is reconciled manually. The five payment methods (cash / visa / mastercard / bank_transfer / cheque) are recorded by an admin clicking "Mark as Paid". Adding a processor brings PCI compliance scope.
- **Refund handling** — handled outside the system. Refunds happen rarely enough that the audit trail in email and bank statements is sufficient.
- **SMS sending** — covered in Section 4.1. Cost and AU compliance considerations.
- **Customer-facing cancel / dispute UI** — covered in Section 1.7. Preserves the recovery phone call.

12. The Glossary

Terms that appear throughout the doc and the codebase, with one-or-two-sentence definitions and the canonical place each lives in code.

Term	Meaning
Lead	A person who's submitted (or had submitted on their behalf) a request for an inspection. The unit of work for the entire pipeline; a single `leads` row carries through from first enquiry to final paid invoice. "Customer" is a synonym used in customer-facing copy; the data lives in the same row.
Inspection	The first site visit. A technician documents mould findings, takes photos, captures climate readings, and quotes a price. Stored in the `inspections` table, one row per visit (typically one per lead).
Job	The remediation work itself, performed after the inspection report is approved by the customer. Bookings carrying `event_type='job'` in `calendar_bookings`; the work performed is captured in the Job Completion form.
Job Completion	The post-job documentation form (10 sections — see Section 2.8). Generates the Job Completion Report PDF. Lives in the `job_completions` table.
AI Summary	The LLM-generated narrative that becomes the body of the inspection report ("What we found", "Treatment plan", "Detailed analysis"). Built by the `generate-inspection-summary` Edge Function via OpenRouter (Gemini family).
AI Review	The admin screen where the AI Summary is reviewed and edited before approval. URL: `/admin/inspection-ai-review/:leadId`. Source: `src/pages/InspectionAIReview.tsx`.
Send-back	When admin returns a submitted inspection to the technician for changes. Implemented as a status change to `inspection_waiting`; the technician's dashboard surfaces a "Revisions" banner. Distinct from "Revision" (a new PDF version).
Revision	A new version of a generated PDF. The `pdf_versions` table is intended to track these but is currently unused — see Section 11.
Archived	A soft-deleted lead. `leads.archived_at` is set to a timestamp; queries filter on `archived_at IS NULL`. Reversible by an admin via the lead list. Distinct from "Voided" (invoice) and "Cancelled" (booking).
Voided	A cancelled invoice. `invoices.status='void'`. The API exists; UI exposure is limited (see Section 3.11).
Cancelled	A cancelled `calendar_bookings` appointment. Different concept from a voided invoice or an archived lead.
Closed / Finished / Not Landed	Three terminal lead statuses. `closed` and `finished` both mean "successfully completed" (legacy and newer name respectively, both still in the enum); `not_landed` means the lead was lost (customer chose a competitor, didn't proceed, etc.).
Activity Log	The human-readable per-lead event timeline. Visible on the Lead Detail page. Stored in the `activities` table; populated by trigger and by application calls.
Audit Log	The immutable database change history (old + new values, acting user). Stored in `audit_logs`, populated by triggers, tamper-resistant by enforcement (no UPDATE or DELETE allowed). No admin UI; queryable only by developers via Supabase Studio.
Email Log	Outbound email delivery records. Stored in `email_logs`; tracks send attempts and Resend's reported status. Bounce / spam status fields exist but are not currently populated (no Resend webhook listener).
Booking	The user-facing term for a scheduled appointment (inspection or job). Stored in `calendar_bookings`, which was renamed from `calendar_events` in migration `20251111000016`. A handful of older comments still reference the old name.
Technician / Inspector	Same person. The role is named `technician` in the `roles` table; the column `inspections.inspector_id` is a legacy of older naming. The user-facing label is "Technician" everywhere in the UI.
Admin	The privileged role with access to the full pipeline, finance, and user management. `roles.name='admin'`. A third role, `manager`, is defined in the table but currently unused.
Hipages	A lead source — the third-party trade-services marketplace. **Not** an integration. The `hipages_lead` status in the lead enum is set manually by an admin when entering a lead that came in via hipages.com.au.
Profile	A user's display data — name, onboarding flag, optional avatar. Stored in the `profiles` table. Distinct from the credentials, which live in Supabase-managed `auth.users`.